

Developing Full Stack Next.js Web Applications

Dr. Jose Annunziato

Table of Contents

Chapter 1 - Building Next.js User Interfaces with HTML	4
1.1 Learning Objectives.....	5
1.2 Setting Up the Development Environment.....	5
1.3 Introduction to HTML.....	9
1.4 Prototyping the React Kambaz User Interface with HTML.....	20
1.5 Committing Code to Source Control.....	33
1.6 Deploying Next.js Projects to the Web.....	36
1.7 Conclusion.....	36
Chapter 2 - Styling Web Pages with CSS	37
2.1 Styling React Components with CSS (Cascading Style Sheets).....	37
2.2 Decorating Documents with React Icons.....	53
2.3 Styling Webpages with the React Bootstrap CSS Library.....	54
2.4 Styling Kambaz with CSS and Bootstrap.....	66
2.6 Delivery.....	79
Chapter 3 - Creating Single Page Applications with React	81
3.1 Learning Objectives.....	81
3.2 Introduction to JavaScript.....	81
3.3 JavaScript Functions.....	86
3.4 JavaScript Data Structures.....	88
3.5 Dynamic Styling.....	98
3.6 Parameterizing Components.....	100
3.7 Rendering a Data Structure.....	104
3.8 Debugging.....	105
3.8 Implementing a Data Driven Kambaz Application.....	106
3.9 Deliverables.....	113
Chapter 4 - Maintaining State in React Applications.....	115
4.1 Learning Objectives.....	115
4.2 Managing State and User Input with Forms.....	115
4.3 Managing Application State with Redux.....	124
4.4 Adding State to the Kambaz User Interface.....	133
4.5 Deliverables.....	151
Chapter 5 - Implementing RESTful Web APIs with Express.js.....	152
5.1 Installing and Configuring an HTTP Web Server.....	152
5.2 Lab Exercises.....	157
5.3 Implementing the Kambaz Node.js HTTP Server.....	181
5.4 Deploying RESTful Web Service APIs to a Public Remote Server.....	204
5.5 Conclusion.....	207
5.6 Deliverables.....	207
Chapter 6 - Integrating React with MongoDB.....	208
6.1 Working with a Local MongoDB Instance.....	209
6.2 Programming with a MongoDB Database.....	211

6.3 Integrating with MongoDB Hosted in Atlas Cloud Service.....	225
6.4 Integrating the Kambaz Web Application with a Database.....	227
6.5 Deliverables.....	246
7 References.....	247

Chapter 4 - Maintaining State in React Applications

In an application, **state** is the collection of data values stored in the various constants, variables and data structures in an application. **Application state** is data that is relevant across the entire application or a significant subset of related components. **Component state** is data that is only relevant to a specific component or a small set of related components. If information is relevant across several or most components, then it should live in the **application state**. If information is relevant only in one component, or a small set of related components, then it should live in the **component state**. For instance, the information about the currently logged in user could be stored in a profile, e.g., **username**, **first name**, **last name**, **role**, **logged in**, etc., and it might be relevant across the entire application. On the other hand, filling out shipping information might only be relevant while checking out, but not relevant anywhere else, so shipping information might best be stored in the **ShippingScreen** or **Checkout** components in the component's state. This chapter introduces how to maintain state at the application level as well as at the component level. The [Redux state management library](#) is described to handle application state, and **React** state hooks are used to manage component state.

4.1 Learning Objectives

By the end of this chapter, you will be able to:

- Understand **state management** in React applications
- Learn how to **handle user input with controlled components**
- Use the **useState** hook to manage component-level state
- Explore **two-way data binding** for form elements
- Implement **React forms** with various input types (text fields, checkboxes, radio buttons, dropdowns)
- Manage **application-wide state** using Redux
- Understand the **Redux store, actions, and reducers**
- Use **Redux Toolkit** to simplify state management
- Connect React components to Redux using **useSelector** and **useDispatch**
- Handle **side effects** with the **useEffect** hook
- Add state management to the **Kambaz user interface**
- Implement **dynamic content rendering** based on application state

4.2 Managing State and User Input with Forms

This section presents **React** examples to program the browser, interact with the user, and generate dynamic HTML. Use the same project you worked on the last chapter. After you work through the examples you will apply the skills while creating a **Kambaz** on your own. Using **IntelliJ**, **VS Code**, or your favorite IDE, open the project you created in previous chapters. **Include all the work in the Labs section as part of your final deliverable.** Do all your work in a new branch called **a4** and deploy it to **Vercel** to a branch deployment of the same name. To get started, create a **Lab4** screen that will host all the exercises in this chapter. Import the component into the **Labs** screen created in an earlier chapter and add a route to navigate to **Lab4**. Style the links using [Bootstrap pills](#) as shown here on the right.

Labs

[Lab 1](#)[Lab 2](#)[Lab 3](#)[Lab 4](#)

4.2.1 Handling User Events

Users interact with the Web application user interface by clicking their mouse, typing at their keyboards, and, on mobile devices, tapping, swiping, and pinching at the screen. As they interact with the graphical user interface, they generate a stream of events that need to be handled by interpreting the user intent, modifying the Web application state, and

rerendering the user interface to reflect the new state and give feedback to the user that their actions are having the intended effect. The next few sections consider the various types of events users generate and how they can be handled.

4.2.1.1 Handling Click Events

The **onClick** attribute can be used to declare a function that handles clicks. The example below calls function **hello** when you click the **Click Hello** button. Note the "use client" at the top of the file. In Next.js, JavaScript files execute on the server by default, and can not interact with the user. User interaction such as mouse clicks, keyboard typing, and dialog windows, all need to interact with the browser and computer hardware, and therefore need to run on the browser, e.g., the client. JavaScript files tagged with **"use client"** do not execute on the server and are instead sent to the browser to execute there. The example below requires users interacting with the mouse or mousepad, and therefore needs to be tagged with **"use client"**. Add the component to **Lab4** and confirm it behaves as expected.

app/Labs/Lab4/ClickEvent.tsx

```
"use client" // declare a function to handle the event
const hello = () => {
  alert("Hello World!");
};
const lifeIs = (good: string) => {
  alert(`Life is ${good}`);
};
export default function ClickEvent() {
  return (
    <div id="wd-click-event">
      <h2>Click Event</h2>
      <button onClick={hello} id="wd-hello-world-click"> // configure the function call
        Hello World!</button>
      <button onClick={() => lifeIs("Good!")} id="wd-life-is-good-click"> // wrap in function if you need to pass parameters
        Life is Good!</button>
      <button onClick={() => { // wrap in {} if you need more than one line of code
        hello(); // calling hello()
        lifeIs("Great!"); // calling lifeIs()
      }} id="wd-life-is-great-click">
        Life is Great!
      </button>
      <hr/>
    </div>
  );
}
```

4.2.1.2 Passing Data when Handling Events

Passing Data on Event

When handling an event, sometimes we need to pass parameters to the function handling the event. Make sure to wrap the function call in a **closure** as shown below. The example below calls **add(2, 3)** when the button is clicked, passing arguments **a** and **b** as **2** and **3**. If you do not wrap the function call inside a closure, you risk creating an infinite loop. Add the component to **Lab4** and confirm it works as expected.

Pass 2 and 3 to add()

app/Labs/Lab4/PassingDataOnEvent.tsx

```
"use client"
const add = (a: number, b: number) => { // function expects a and b
  alert(`${a} + ${b} = ${a + b}`);
};
export default function PassingDataOnEvent() {
  return (
    <div id="wd-passing-data-on-event">
      <h2>Passing Data on Event</h2>
      <button onClick={() => add(2, 3)} // use this syntax
        // onClick={add(2, 3)} // and not this syntax.
        className="btn btn-primary" // Otherwise you risk
        id="wd-pass-data-click"> // creating an infinite loop
    </div>
  );
}
```

```

    Pass 2 and 3 to add()
  </button>
</hr>
</div>
);}

```

4.2.1.3 Passing Functions as Parameters

In JavaScript, functions can be treated as any other constant or variable, including passing them as parameters to other functions. The example below passes function **sayHello** to component **PassingFunctions**. When the button is clicked, **sayHello** is invoked.

Passing Functions

Invoke the Function

app/Labs/Lab4/PassingFunctions.tsx

```

"use client"
export default function PassingFunctions( // function passed in as a parameter
  { theFunction }: { theFunction: () => void } ) {
  return (
    <div>
      <h2>Passing Functions</h2> // invoking function with no arguments
      <button onClick={theFunction} className="btn btn-primary">
        Invoke the Function
      </button>
      <hr/>
    </div>
  );}

```

Include the component in **Lab4**, declare a **sayHello** callback function, pass it to the **PassingFunctions** component, and confirm it works as expected.

app/Labs/Lab4/page.tsx

```

"use client" // add "use client"
import PassingFunctions from "../PassingFunctions"; // import the component
export default function Lab4() {
  function sayHello() { // implement callback function
    alert("Hello");
  }
  return (
    <div id="wd-passing-functions">
      <h2>Lab 4</h2>
      ...
      <PassingFunctions theFunction={sayHello} /> // pass callback function as a parameter
    </div>
  );}

```

4.2.2 Managing Component State

Web applications implemented with React can be considered as a set of functions that transform a set of data structures into an equivalent user interface. The collection of data structures and values are often referred to as an application **state**. So far we have explored React applications that transform a static data set, or state, into a static user interface. We will now consider how the state can change over time as users interact with the user interface and how these state changes can be represented in a user interface.

Users interact with an application by clicking, dragging, and typing with their mouse and keyboard, filling out forms, clicking buttons, and scrolling through data. As users interact with an application they create a stream of events that can be handled by a set of event handling functions, often referred to as **controllers**. Controllers handle user events and convert them into changes in the application's state. Applications render application state changes into corresponding

changes in the user interface to give users feedback of their interactions. In Web applications, user interface changes consist of changes to the DOM.

4.2.2.1 Use State Hook

Updating the DOM with JavaScript is slow and can degrade the performance of Web applications. React optimizes the process by creating a **virtual DOM**, a more compact and efficient version of the real DOM. When React renders something on the screen, it first updates the virtual DOM, and then converts these changes into updates to the actual DOM. To avoid unnecessary and slow updates to the DOM, React only updates the real DOM if there have been changes to the virtual DOM. We can participate in this process of state change and DOM updates by using the **useState** hook. The **useState** hook is used to declare **state** variables that we want to affect the DOM rendering. The syntax of the **useState** hook is shown below.

```
const [stateVariable, setStateVariable] = useState(initialStateValue);
```

The **useState** hook takes as argument the initial value of a **state variable** and returns an array whose first item consists of the initialized state variable, and the second item is a **mutator** function that allows updating the state variable. The array destructor syntax is commonly used to bind these items to local constants as shown above. The mutator function not only changes the value of the state variable, but it also notifies React that it should check if the state has caused changes to the virtual DOM and therefore make changes to the actual DOM. The following exercises introduce various use cases of the **useState**.

4.2.2.2 Integer State Variables

To illustrate the point of the **virtual DOM** and how changes in state affect changes in the actual DOM, let's implement the simple **Counter** component as shown below. A **count** variable is initialized and then rendered successfully on the screen. Buttons **Up** and **Down** successfully update the **count** variable as evidenced in the console, but the changes fail to update the DOM as desired. This happens because as far as React is concerned, there has been no changes to the virtual DOM, and therefore no need to update the actual DOM.

app/Labs/Lab4/Counter.tsx

```
import { useState } from "react";
export default function Counter() {
  let count = 7;
  console.log(count);
  return (
    <div id="wd-counter-use-state">
      <h2>Counter: {count}</h2>
      <button
        onClick={() => { count++; console.log(count); }}
        id="wd-counter-up-click">Up</button>
      <button
        onClick={() => { count--; console.log(count); }}
        id="wd-counter-down-click">Down</button>
      <hr/></div>);
  // declare and initialize
  // a variable. print changes
  // of the variable to the console
  // render variable
  // variable updates on console
  // but fails to update the DOM as desired
```

Counter: 7



Figure 4.2.2.2a - Integer State Variables

For the DOM to be updated as expected, we need to tell React that changes to a particular variable is indeed relevant to changes in the DOM. To do this, use the **useState** hook to declare the state variable, and update it using the mutator function as shown below. Now changes to the state variable are represented as changes in the DOM. Implement the **Counter** component, import it in **Lab4** and confirm it works as expected. Do the same with the rest of the exercises that follow.

app/Labs/Lab4/Counter.tsx

```
import { useState } from "react"; // import useState
export default function Counter() {
  //let count = 7;
  const [count, setCount] = useState(7); // create and initialize
  console.log(count); // state variable
  return (
    <div>
      <h2>Counter: {count}</h2> // render state variable
      <button onClick={() => setCount(count + 1)} // handle events and update
        id="wd-counter-up-click">Up</button> // state variable with mutator
      <button onClick={() => setCount(count - 1)} // now updates to the state
        id="wd-counter-down-click">Down</button> // state variable do update the
    </div>); // DOM as desired
```

4.2.2.3 Boolean State Variables

The **useState** hook works with all JavaScript data types and structures including **booleans**, **integers**, **strings**, **numbers**, **arrays**, and **objects**. The exercise below illustrates using the **useState** hook with **boolean** state variables. The variable is used to hide or show a DIV as well as render a checkbox as checked or not. Also note the use of **onChange** in the checkbox to set the value of state variable.

app/Labs/Lab4/BooleanStateVariables.tsx

```
import { useState } from "react"; // import useState
export default function BooleanStateVariables() {
  const [done, setDone] = useState(true); // declare and initialize
  return ( // boolean state variable
    <div id="wd-boolean-state-variables">
      <h2>Boolean State Variables</h2>
      <p>{done ? "Done" : "Not done"}</p> // render content based on
      <label className="form-control"> // boolean state variable value
        <input type="checkbox" checked={done} // change state variable value
          onChange={() => setDone(!done)} /> Done // when handling events like
      </label> // clicking a checkbox
      {done && <div className="alert alert-success"> // render content based on
        Yay! you are done</div>} // boolean state variable value
    </div>);
```

Boolean State Variables

Done

☒ Done

Yay! you are done

Figure 4.2.2.3 - Boolean State Variables

4.2.2.4 String State Variables

The **StringStateVariables** exercise below illustrates using **useState** with string state variables. The input field's **value** is initialized to the **firstName** state variable. The **onChange** attribute invokes the **setFirstName** mutator function to update the state variable. The **e.target.value** contains the value of the input field and is used to update the current value of the state variable.

app/Labs/Lab4/StringStateVariables.tsx

```
import { useState } from "react"; // import useState
import { FormControl } from "react-bootstrap";
export default function StringStateVariables() {
  const [firstName, setFirstName] = useState("John"); // declare and
  return ( // initialize
    <div> // state variable
      <h2>String State Variables</h2>
      <p>{firstName}</p> // render string
      <FormControl // state variable
        defaultValue={firstName} // initialize a
        onChange={(e) => setFirstName(e.target.value)}> // text input field with the state variable
      </div>); // update the state variable at each key stroke
```

String State Variables

John Doe

Figure 4.2.2.4 - String State Variables

4.2.2.5 Date State Variables

The **DateStateVariable** component illustrates how to work with date state variables. The **startDate** state variable is initialized to the current date using **new Date()** which has the string representation as shown here on the right. The **dateObjectToHtmlDateString** function can convert a **Date** object into the **YYYY-MM-DD** format expected by the HTML date input field. The function is used to initialize and set the date field's **value** attribute so it matches the expected format. Changes in date field are handled by the **onChange** attribute which updates the new date using the **setStartDate** mutator function.

app/Labs/Lab4/DateStateVariable.tsx

```
import { useState } from "react"; // import useState
import { FormControl } from "react-bootstrap";
export default function DateStateVariable() {
  const [startDate, setStartDate] = useState(new Date()); // declare and initialize with today's date
  const dateObjectToHtmlDateString = (date: Date) => { // utility function to convert date object
    return `${date.getFullYear()}-${date.getMonth() + 1 < 10 ? 0 : ""}${ // to YYYY-MM-DD format for HTML date
      date.getMonth() + 1
    }-${date.getDate() + 1 < 10 ? 0 : ""}${date.getDate() + 1}`; // picker
  };
  return (
    <div id="wd-date-state-variables">
      <h2>Date State Variables</h2>
      <h3>{JSON.stringify(startDate)}</h3> // display raw date object
      <h3>{dateObjectToHtmlDateString(startDate)}</h3> // display in YYYY-MM-DD format for input
      <FormControl // of type date
        type="date"
        defaultValue={dateObjectToHtmlDateString(startDate)} // set HTML input type to date
        onChange={(e) => setStartDate(new Date(e.target.value))}> // update when you change the date with
      </div>); // the date picker
```

Date State Variables
"2023-10-09T01:57:28.439Z"
2023-10-09

10/09/2023

Figure 4.2.2.5 - Date State Variables

4.2.2.6 Object State Variables

The **ObjectStateVariable** component below demonstrates how to work with object state variables. We declare **person** object state variable with initial property values **name** and **age**. The object is rendered on the screen using **JSON.stringify** to see the changes in real time. Two value of two input fields are initialized to the object's **person.name** string property and the object's **person.age** number property. As the user types in the input fields, the **onChange** attribute passes the events to update the object's property using the **setPerson** mutator functions. The object is updated by creating new objects copied from the previous object value using the spreader operator (**...person**), and then overriding the **name** or **age** property with the **target.value**.

app/Labs/Lab4/ObjectStateVariable.tsx

```
import { useState } from "react";
import { FormControl } from "react-bootstrap";
export default function ObjectStateVariable() {
  const [person, setPerson] = useState({ name: "Peter", age: 24 }); // declare and initialize object state
  // variable with multiple fields

  return (
    <div>
      <h2>Object State Variables</h2>
      <pre>{JSON.stringify(person, null, 2)}</pre> // display raw JSON
      <FormControl // initialize input field with an object's
        // field value
        defaultValue={person.name} // update field as user types. copy old
        onChange={(e) => setPerson({ ...person, name: e.target.value })} // object, override specific field with new
        // value
      </>
      <FormControl // update field as user types. copy old
        // object,
        defaultValue={person.age} type="number" // override specific field with new value
        onChange={(e) => setPerson({ ...person,
          age: parseInt(e.target.value) })}
      </>
    </div>
  );
}
```

Object State Variables
{
 "name": "Russell Peters",
 "age": "53"
}

Russell Peters
53

Figure 4.2.2.6 - Object State Variables

4.2.2.7 Array State Variables

The **ArrayStateVariable** component below demonstrates how to work with **array** state variables. An array of integers is declared as a state variable and function **addElement** and **deleteElement** are used to add and remove elements to and from the array. We render the array as a map of line items in an unordered list. We render the array's value and a **Delete** button for each element. Clicking the **Delete** button calls the **deleteElement** function which passes the **index** of the element we want to remove. The **deleteElement** function computes a new array filtering out the element by its position and updating the **array** state variable to contain a new array without the element we filtered out. Clicking the **Add Element** button invokes the **addElement** function which computes a new array with a copy of the previous **array** spread at the beginning of the new array, and adding a new random element at the end of the array. Add Bootstrap classes so the output renders as shown.

app/Labs/Lab4/ArrayStateVariable.tsx

```
import { useState } from "react"; // import useState
export default function ArrayStateVariable() {
  const [array, setArray] = useState([1, 2, 3, 4, 5]); // declare array state
  const addElement = () => { // event handler appends
    setArray([...array, Math.floor(Math.random() * 100)]); // random number at end of
  }; // array
  const deleteElement = (index: number) => { // event handler removes
    setArray(array.filter((item, i) => i !== index)); // element by index
  };
  return (
    <div id="wd-array-state-variables">
      <h2>Array State Variable</h2>
      <button onClick={addElement}>Add Element</button> // button calls addElement
      <ul> // to append to array
        {array.map((item, index) => ( // iterate over array items
          <li key={index}> {item}
            <button onClick={() => deleteElement(index)}> // render item's value
              Delete</button> // button to delete element
            </li>))} // by its index
        </ul><hr/></div>);}
```

Array State Variable

Add Element	
1	Delete
2	Delete
3	Delete
4	Delete
5	Delete

Figure 4.2.2.7 - Array State Variables

4.2.2.8 Sharing State Between Components

State can be shared between components by passing references to state variables and/or functions that update them. The example below demonstrates a **ParentStateComponent** sharing **counter** state variable and **setCounter** mutator function with **ChildStateComponent** by passing it references to **counter** and **setCounter** as attributes.

app/Labs/Lab4/ParentStateComponent.tsx

```
import { useState } from "react";
import ChildStateComponent from "../ChildStateComponent";
export default function ParentStateComponent() {
  const [counter, setCounter] = useState(123);
  return (
    <div>
      <h2>Counter {counter}</h2>
      <ChildStateComponent
        counter={counter}
        setCounter={setCounter} />
      <hr/>
    </div>
  );
};
```

The **ChildStateComponent** can use references to **counter** and **setCounter** to render the state variable and manipulate it through the mutator function. Import **ParentStateComponent** into **Lab4** and confirm it works as expected.

app/Labs/Lab4/ChildStateComponent.tsx

```
export default function ChildStateComponent({
  counter,
  setCounter
}: {
  counter: number;
  setCounter: (counter: number) => void;
}) {
  return (
    <div id="wd-child-state">
      <h3>Counter {counter}</h3>
      <button onClick={() => setCounter(counter + 1)} id="wd-increment-child-state-click">
        Increment</button>
      <button onClick={() => setCounter(counter - 1)} id="wd-decrement-child-state-click">
        Decrement</button>
    </div>
  );
};
```

4.2.3 Encoding State in URL

Components can pass data to each other through attributes or as child content in the component's body. Pages can also pass data to each other by encoding it in the URL that navigates to the page. URLs are made up of the following parts: <http://example.com/path/to/the/page?optional=data&encoded=in-query>

- [http://](#) - the **protocol**
- [example.com](#) - the **domain** or IP address of the server
- [/path/to/the/page](#) - the real or virtual **path** to an actual document or resolved to some virtual computed content
- [?optional=data&encoded=in-query](#) - optional **query string** with name value pairs delimited with ampersand (&)

There are two strategies to encode data in the URL: **query parameters** and **path parameters**.

4.2.3.1 Query Search Parameters

The **QueryCalculator** page below demonstrates how to read data from the **query string** using Next.js **useSearchParams** hook. The **useSearchParams** hook decodes the names and values from the URL and returns an object map where the keys are the names of the parameters and the values are the values are the parameters. This strategy is great for optional filters, search terms, pagination, or any non-structural data.

app/Labs/Lab4/url-encoding/query-params/page.tsx

```
"use client";
```

```
import { useSearchParams } from "next/navigation";

export default function QueryCalculator() {
  const searchParams = useSearchParams();

  const aRaw = searchParams.get("a") || "0";
  const bRaw = searchParams.get("b") || "0";

  const a = parseFloat(aRaw);
  const b = parseFloat(bRaw);
  const sum = a + b;

  return (
    <div style={{ padding: 40 }}>
      <h1>Calculator - Query Parameters</h1>

      Raw query values (already decoded by Next.js):
      <p>
        a = <code>{aRaw}</code>
      </p>
      <p>
        b = <code>{bRaw}</code>
      </p>

      <h2 style={{ color: "green" }}>Sum = {sum}</h2>
    </div>
  );
}
```

4.2.3.2 Path Parameters

The **PathCalculator** page below demonstrates the same capabilities as the **QueryCalculator** page, but using **path parameters** where data is encoded as part of the URL path instead of name value pairs after the ? at the end of the URL. Here we use the **useParams** hook instead which decodes the parameters from the path. Note that the names of the parameters are encoded as part of the physical name of the directory path, e.g., **[a]** and **[b]**.

app/labs/Lab4/url-encoding/path-params/[a]/[b]/page.tsx

```
"use client";

import { useParams } from "next/navigation";

export default function PathCalculator() {
  const params = useParams();

  const aRaw = params.a as string;
  const bRaw = params.b as string;

  const a = parseFloat(aRaw);
  const b = parseFloat(bRaw);
  const sum = a + b;

  return (
    <div style={{ padding: 40 }}>
      <h1>Calculator - Path Parameters</h1>
      Raw path segments (already decoded by Next.js):
      <p>
        a = <code>{aRaw}</code>
      </p>
      <p>
        b = <code>{bRaw}</code>
      </p>
      <h2 style={{ color: "green" }}>Sum = {sum}</h2>
    </div>
  );
}
```

The pages above read data from the URL using either query or path parameters. The **UrlEncoding** component below illustrates how the parameters can be encoded into the URLs using query and path parameters and then navigate to either the **QueryCalculator** or **PathCalculator** accordingly.

app/Labs/Lab4/query-parameters/index.tsx

```
"use client";

import { useState } from "react";
import { useRouter } from "next/navigation";
import Link from "next/link";

export default function UrlEncoding() {
  const [a, setA] = useState("5");
  const [b, setB] = useState("10");
  const router = useRouter();
  const baseUrl = "/labs/lab4/url-encoding";

  // Programmatic navigation (using router.push)
  const goToQueryVersion = () => {
    const params = new URLSearchParams();
    params.set("a", a);
    params.set("b", b);
    router.push(`${baseUrl}/query-params?${params.toString()}`);
  };

  const goToPathVersion = () => {
    const safeA = encodeURIComponent(a);
    const safeB = encodeURIComponent(b);
    router.push(`${baseUrl}/path-params/${safeA}/${safeB}`);
  };

  return (
    <div style={{ padding: 40, maxWidth: 600 }}>
      <h2>Addition Calculator</h2>
      <p>
        Enter two numbers and navigate using either buttons (programmatic) or
        links (declarative):
      </p>
      <input
        type="number"
        value={a}
        onChange={(e) => setA(e.target.value)}
        className="form-control"
      />
      <input
        type="number"
        value={b}
        onChange={(e) => setB(e.target.value)}
        className="form-control"
      />
      <h4>Programmatic navigation (using router.push):</h4>
      <button onClick={goToQueryVersion} className="btn btn-success w-100">
        {a} + {b} → Query Params (programmatic)
      </button>
      <button onClick={goToPathVersion} className="btn btn-success w-100 mt-2">
        {a} + {b} → Path Params (programmatic)
      </button>
      <h4>Declarative navigation (using <Link>):</h4>
      <Link
        href={` ${baseUrl}/query-params?a=${encodeURIComponent(a)}&b=${encodeURIComponent(b)}`}
        className="btn btn-primary w-100"
      >
        {a} + {b} → Query Params (Link)
      </Link>
      <Link
        href={` ${baseUrl}/path-params/${encodeURIComponent(a)}/${encodeURIComponent(b)}`}
        className="btn btn-primary w-100 mt-2"
      >
        {a} + {b} → Path Params (Link)
      </Link>
    </div>
  );
}
```

```
    </div>
  );
}
```

4.3 Managing Application State with Redux

The **useState** hook is used to maintain the state within a component. State can be shared across components by passing references to state variables and mutators to other components. Although this approach is sufficient as a general approach to share state among multiple components, it is fraught with challenges when building larger, more complex applications. The downside of using **useState** across multiple components is that it creates an explicit dependency between these components, making it hard to refactor components adapting to changing requirements. The solution is to eliminate the dependency using libraries such as [Redux](#). This section explores the Redux library to manage state that is meant to be used across a large set of components, and even an entire application. We'll keep using **useState** to manage state within individual components, but use Redux to manage Application level state. To learn about redux, let's create a redux examples component that will contain several simple redux examples. Create an **page.tsx** file under **app/labs/lab4/redux/page.tsx** as shown below. Import the new redux examples component into the Lab 4 component so we can see how it renders as we add new examples. Reload the browser and confirm the new component renders as expected.

app/labs/Lab4/redux/page.tsx

```
export default function ReduxExamples() {
  return (
    <div>
      <h2>Redux Examples</h2>
    </div>
  );
}
```

app/Labs/Lab4/page.tsx

```
import Link from "next/link";
export default function Lab4() {
  return (
    <div>
      <h2>Lab 4</h2>
      <Link href="/.redux">Redux Examples</Link>
    </div>
  );
}
```

4.3.1 Installing Redux

As mentioned earlier we will be using [the Redux state management library](#) to handle application state. To install **Redux**, type the following at the command line from the root folder of your application.

```
npm install redux --save
```

After redux has installed, install **react-redux** and the redux **toolkit**, the libraries that integrate **redux** with **React**. At the command line, type the following commands.

```
npm install react-redux --save
npm install @reduxjs/toolkit --save
```

4.3.2 Create a Hello World Redux component

To learn about Redux, let's start with a simple Hello World example. Instead of maintaining state within any particular component, Redux declares and manages state in separate **reducers** which then **provide** the state to the entire application. Create **helloReducer** as shown below maintaining a state that consists of just a **message** state string initialized to **Hello World**.

app/Labs/Lab4/redux/hello/helloReducer.ts

```
import { createSlice } from "@reduxjs/toolkit";
const initialState = {
  message: "Hello World",
};
const helloSlice = createSlice({
  name: "hello",
  initialState,
  reducers: {},
});
export default helloSlice.reducer;
```

Application state can maintain data from various components or screens across an entire application. Each would have a separate reducer that can be combined into a single **store** where reducers come together to create a complex, application wide state. The **store.ts** below demonstrates adding the **helloReducer** to the store. Later exercises and the **Kambaz** section will add additional reducers to the store.

app/Labs/Lab4/store.ts

```
import { configureStore } from "@reduxjs/toolkit";
import helloReducer from "../redux/hello/helloReducer";
const store = configureStore({
  reducer: { helloReducer } });
export type RootState = ReturnType<typeof store.getState>;
export default store;
```

The application state can then be shared with the entire Web application by wrapping it with a **Provider** component that makes the state data in the **store** available to all components within the **Provider's** body.

app/Labs/Lab4/page.tsx

```
...
import store from "../store";
import { Provider } from "react-redux";
export default function Lab4() {
  return (
    <Provider store={store}>
      <div>
        <h2>Lab 4</h2>
        <ReduxExample/>
        ...
      </div>
    </Provider>
  );
}
```

Components within the body of the **Provider** can then **select** the state data they want using the **useSelector** hook as shown below. Add the **HelloRedux** component to **ReduxExamples** and confirm it renders as shown below.

app/Labs/Lab4/redux/hello/index.tsx

```
import { useSelector, useDispatch } from "react-redux";
import { RootState } from "../../store";
export default function HelloRedux() {
  const { message } = useSelector((state: RootState) => state.helloReducer);
  return (
    <div id="wd-hello-redux">
      <h3>Hello Redux</h3>
      <h4>{message}</h4> <hr />
    </div>
  );
}
```


Redux Examples

Hello Redux

Hello World

Figure 4.3.2 - Create a Hello World Redux component

4.3.3 Counter Redux - Dispatching Events to Reducers

To practice with Redux, let's reimplement the **Counter** component using Redux. First create **counterReducer** responsible for maintaining the counter's state. Initialize the state variable **count** to 0, and reducer function **increment** and **decrement** can update the state variable by manipulating their **state** parameter that contain state variables as shown below.

app/Labs/Lab4/redux/CounterRedux/counterReducer.tsx

```
import { createSlice } from "@reduxjs/toolkit";
const initialState = {
  count: 0,
};
const counterSlice = createSlice({
  name: "counter",
  initialState,
  reducers: {
    increment: (state) => {
      state.count = state.count + 1;
    },
    decrement: (state) => {
      state.count = state.count - 1;
    },
  },
});
export const { increment, decrement } = counterSlice.actions;
export default counterSlice.reducer;
```

Add the **counterReducer** to the **store** as shown below to make the counter's state available to all components within the body of the **Provider**.

app/Labs/Lab4/store.ts

```
import { configureStore } from "@reduxjs/toolkit";
import helloReducer from "../redux/hello/helloReducer";
import counterReducer from "../redux/CounterRedux/counterReducer";
const store = configureStore({
  reducer: {
    helloReducer,
    counterReducer,
  },
});
export type RootState = ReturnType<typeof store.getState>;
export default store;
```

The **CounterRedux** component below can then **select** the **count** state from the store using the **useSelector** hook. To invoke the reducer function **increment** and **decrement** use a **dispatch** function obtained from a **useDispatch** function as shown below. Add **CounterRedux** to **ReduxExamples** and confirm it works as expected.

app/Labs/Lab4/redux/CounterRedux/index.tsx

```
import { useSelector, useDispatch } from "react-redux";
import { increment, decrement } from "../counterReducer";
```

```
import { RootState } from "../../store";
export default function CounterRedux() {
  const { count } = useSelector((state: RootState) => state.counterReducer);
  const dispatch = useDispatch();
  return (
    <div id="wd-counter-redux">
      <h2>Counter Redux</h2>
      <h3>{count}</h3>
      <button onClick={() => dispatch(increment())} id="wd-counter-redux-increment-click"> Increment </button>
      <button onClick={() => dispatch(decrement())} id="wd-counter-redux-decrement-click"> Decrement </button>
      <hr/>
    </div>
  );
};
```

Counter Redux

5



Figure 4.3.3 - Counter Redux - Dispatching Events to Reducers

4.3.4 Passing Data to Reducers

Now let's explore how the user interface can pass data to reducer functions. Create a reducer that can keep track of the arithmetic addition of two parameters. When we call **add** reducer function below, the parameters are encoded as an object into a **payload** property found in the **action** parameter passed to the reducer function. Functions can extract parameters **a** and **b** as **action.payload.a** and **action.payload.b** and then use the parameters to update the **sum** state variable.

app/Labs/Lab4/redux/AddRedux/addReducer.tsx

```
import { createSlice } from "@reduxjs/toolkit";
const initialState = {
  sum: 0,
};
const addSlice = createSlice({
  name: "add",
  initialState,
  reducers: {
    add: (state, action) => {
      state.sum = action.payload.a + action.payload.b;
    },
  },
});
export const { add } = addSlice.actions;
export default addSlice.reducer;
```

Add the new reducer to the store so it's available throughout the application as shown below.

app/Labs/Lab4/store.ts

```
import { configureStore } from "@reduxjs/toolkit";
import helloReducer from "../redux/hello/helloReducer";
import counterReducer from "../redux/CounterRedux/counterReducer";
import addReducer from "../redux/AddRedux/addReducer";
const store = configureStore({
  reducer: {
    helloReducer,
```

```

    counterReducer,
    addReducer,
  },
});
export type RootState = ReturnType<typeof store.getState>;
export default store;

```

To tryout the new reducer, import the **add** reducer function as shown in the **AddRedux** component below. Maintain the values of **a** and **b** as local component state variables, and then pass them to **add** as a single object. Add **AddRedux** to **ReduxExamples** to confirm it works as expected.

app/Labs/Lab4/redux/AddRedux/index.tsx

```

import { useSelector, useDispatch } from "react-redux";           // to read/write to reducer
import { useState } from "react";                                // to maintain a and b parameters in UI
import { add } from "../addReducer";
import { RootState } from "../../store";
export default function AddRedux() {
  const [a, setA] = useState(12);                                // a and b state variables to edit
  const [b, setB] = useState(23);                                // parameters to add in the reducer
  const { sum } = useSelector((state: RootState) =>               // read the sum state variable from the reducer
    state.addReducer);                                           // dispatch to call add redux function
  const dispatch = useDispatch();
  return (
    <div className="w-25" id="wd-add-redux">                       // render local state variables a and b, as well
      <h1>Add Redux</h1>                                           // as application state variable sum
      <h2>{a} + {b} = {sum}</h2>
      <FormControl type="number" defaultValue={a}                 // update the local component state variable a
        onChange={(e) => setA(parseInt(e.target.value))}> />
      <FormControl type="number" defaultValue={b}                 // update the local component state variable b
        onChange={(e) => setB(parseInt(e.target.value))}> />
      <Button id="wd-add-redux-click"                               // on click, call add reducer function to
        onClick={() => dispatch(add({ a, b })))}>                 // compute the arithmetic addition of a and b,
        Add Redux                                                 // and store it in application state
      </Button>                                                    // variable sum
      <hr/>
    </div>
  );
}

```

4.3.5 Implementing a Todo List with Redux

Let's practice using local component state as well as application level state to implement a simple **Todo List** component. First we'll implement the component using only component state with **useState** which will limit the todos to only available within the **Todo List**. We'll then add application state support to demonstrate how the todos can be shared with any component or screen in the application. Create the **TodoList** component as shown below. Add **Bootstrap** classes to style the todos as shown here on the right.

app/Labs/Lab4/redux/todos/ToDoList.tsx

```

import { useState } from "react";                                // import useState
export default function ToDoList() {
  const [todos, setTodos] = useState([                           // create todos array state variable
    { id: "1", title: "Learn React" },                           // initialize with 2 todo objects
    { id: "2", title: "Learn Node" } ]);
  const [todo, setTodo] = useState({ id: "-1", title: "Learn Mongo" }); // create todo state variable object
  const addTodo = (todo: any) => {                                // event handler to add new todo
    const newTodos = [ ...todos, { ...todo,                     // spread existing todos, append new todo,
      id: new Date().getTime().toString() } ];                  // override id
    setTodos(newTodos);                                           // update todos
    setTodo({ id: "-1", title: "" });                             // clear the todo
  };
  const deleteTodo = (id: string) => {                           // event handler to remove

```

```

const newTodos = todos.filter((todo) => todo.id !== id);           // todo by their ID
setTodos(newTodos);
};
const updateTodo = (todo: any) => {                               // event handler to
  const newTodos = todos.map((item) =>                           // update todo by
    (item.id === todo.id ? todo : item));                       // replacing todo
  setTodos(newTodos);                                           // by their ID
  setTodo({id: "-1", title: ""});
};
return (
  <div>
    <h2>Todo List</h2>
    <ListGroup>
      <ListGroupItem>
        <Button onClick={() => addTodo(todo)}                    // add todo button
          id="wd-add-todo-click"> Add </Button>
        <Button onClick={() => updateTodo(todo)}                 // update todo button
          id="wd-update-todo-click"> Update </Button>
        <FormControl value={todo.title}                         // input field to update todo's title
          onChange={(e) => setTodo({ ...todo, title: e.target.value })}/> // for every keystroke
      </ListGroupItem>
      {todos.map((todo) => (
        <ListGroupItem key={todo.id}>
          <Button onClick={() => deleteTodo(todo.id)}             // update the todo's title,
            id="wd-delete-todo-click"> Delete </Button>           // but copy old values first
          <Button onClick={() => setTodo(todo)}                   // render all todos
            id="wd-set-todo-click"> Edit </Button>                // as line items
          {todo.title}
        </ListGroupItem>
      ))}
    </ListGroup><hr/>
  </div>);}

```

Todo List

Learn Mongo	Update	Add
Learn React	Edit	Delete
Learn Node	Edit	Delete

Figure 4.3.5 - Implementing a Todo List with Redux

4.3.5.1 Breaking up Large Components

Let's break up the **TodoList** component into several smaller components: **TodoItem** and **TodoForm**. **TodoItem** shown below breaks out the line items that render the todo's title, and **Delete** and **Edit** buttons. The component accepts references to the **todo** object, as well as **deleteTodo** and **setTodo** functions.

app/Labs/Lab4/redux/todos/ToDoItem.tsx

```

export default function TodoItem({ todo, deleteTodo, setTodo }: { // breaks out todo item
  todo: { id: string; title: string };                          // todo to render
  deleteTodo: (id: string) => void;                             // event handler to remove todo
  setTodo: (todo: { id: string; title: string }) => void;       // event handler to select todo
}) {
  return (
    <ListGroupItem key={todo.id}>

```

```

<Button onClick={() => deleteTodo(todo.id)} // invoke delete todo with ID
  id="wd-delete-todo-click"> Delete </Button>
<Button onClick={() => setTodo(todo)} // invoke select todo
  id="wd-set-todo-click"> Edit </Button>
{todo.title} </ListGroupItem>);} // render todo's title

```

Similarly we'll break out the form to **Create** and **Update** todos into component **TodoForm** shown below. Parameters **todo**, **setTodo**, **addTodo**, and **updateTodo**, to maintain dependencies between the **TodoList** and **TodoForm** component.

app/Labs/Lab4/redux/todos/TodoForm.tsx

```

export default function TodoForm({ todo, setTodo, addTodo, updateTodo }: { // breaks out todo form
  todo: { id: string; title: string }; // todo to be added or edited
  setTodo: (todo: { id: string; title: string }) => void; // event handler to update todo's title
  addTodo: (todo: { id: string; title: string }) => void; // event handler to add new todo
  updateTodo: (todo: { id: string; title: string }) => void; // event handler to update todo
}) {
  return (
    <ListGroupItem>
      <Button onClick={() => addTodo(todo)} // invoke add new todo
        id="wd-add-todo-click"> Add </Button> // invoke update todo
      <Button onClick={() => updateTodo(todo)} // input field to update
        id="wd-update-todo-click"> Update </Button> // todo's title
      <FormControl value={todo.title} // update title on each key stroke
        onChange={ (e) => setTodo({ ...todo, title: e.target.value }) } />
    </ListGroupItem>
  );}

```

Now we can replace the form and todo items in the **TodoList** component as shown below. Add the **TodoList** component to **Lab4** and confirm it works as expected.

app/Labs/Lab4/redux/todos/TodoList.tsx

```

import TodoForm from "../TodoForm"; // import TodoForm
import TodoItem from "../TodoItem"; // import TotoItem
export default function TodoList() {
  ...
  return (
    <div id="wd-todo-list-redux">
      <h2>Todo List</h2>
      <ListGroup>
        <TodoForm
          todo={todo} // TodoForm breaks out form to add or update todo
          setTodo={setTodo} // pass state variables and
          addTodo={addTodo} // event handlers
          updateTodo={updateTodo}/> // so component
        {todos.map((todo) => ( // can communicate with TodoList's data and functions
          <TodoItem
            todo={todo} // TodoItem breaks out todo item
            deleteTodo={deleteTodo} // pass state variables and
            setTodo={setTodo} /> // event handlers to
          )
        )} // communicate with TodoList's data and functions
      </ListGroup>
    </div>);}

```

4.3.5.2 Todos Reducer

Although the **TodoList** component might work as expected and it might be all we would need, it's implementation makes it difficult to share the local state data (the todos) outside its context with other components or screens. For instance, how would we go about accessing and displaying the todos, say, in the **Lab3** component or **Kambaz**? We would have to move the todos state variable and mutator functions to a component that is parent to both the **Lab3** component and the **TodoList** component, e.g., **Labs** or even **App**.

Instead, let's move the state and functions from the **ToDoList** component to a reducer and store so that the todos can be accessed from anywhere within the **Labs**. Create **todosReducer** as shown below, moving the **todos** and **todo** state variables to the reducer's **initialState**. Also move the **addTodo**, **deleteTodo**, **updateTodo**, and **setTodo** functions into the **reducers** property, reimplementing them to use the **state** and **action** parameters of the new reducer functions.

app/Labs/Lab4/redux/todos/todosReducer.ts

```
import { createSlice } from "@reduxjs/toolkit";
const initialState = {
  todos: [
    { id: "1", title: "Learn React" },
    { id: "2", title: "Learn Node" },
  ],
  todo: { title: "Learn Mongo" },
};
const todosSlice = createSlice({
  name: "todos",
  initialState,
  reducers: {
    addTodo: (state, action) => {
      const newTodos = [
        ...state.todos,
        { ...action.payload, id: new Date().getTime().toString() },
      ];
      state.todos = newTodos;
      state.todo = { title: "" };
    },
    deleteTodo: (state, action) => {
      const newTodos = state.todos.filter((todo) => todo.id !== action.payload);
      state.todos = newTodos;
    },
    updateTodo: (state, action) => {
      const newTodos = state.todos.map((item) =>
        item.id === action.payload.id ? action.payload : item
      );
      state.todos = newTodos;
      state.todo = { title: "" };
    },
    setTodo: (state, action) => {
      state.todo = action.payload;
    },
  },
});
export const { addTodo, deleteTodo, updateTodo, setTodo } = todosSlice.actions;
export default todosSlice.reducer;
```

Add the new **todosReducer** to the **store** so that it can be provided to the rest of the **Labs**.

app/Labs/Lab4/store.ts

```
import { configureStore } from "@reduxjs/toolkit";
import helloReducer from "../redux/hello/helloReducer";
import counterReducer from "../redux/CounterRedux/counterReducer";
import addReducer from "../redux/AddRedux/addReducer";
import todosReducer from "../redux/todos/todosReducer";
const store = configureStore({
  reducer: {
    helloReducer,
    counterReducer,
    addReducer,
    todosReducer,
  },
});
export type RootState = ReturnType<typeof store.getState>;
export default store;
```

Now that we've moved the state and mutator functions to the **todosReducer**, refactor the **TodoForm** component to use the reducer functions instead of the parameters. Also select the **todo** from the reducer state, instead of the **todo** parameter.

app/Labs/Lab4/redux/todos/ToDoForm.tsx

```
import React from "react";
import { useSelector, useDispatch } from "react-redux";
import { addTodo, updateTodo, setTodo } from "../todosReducer";
import { RootState } from "../../store";
export default function ToDoForm(
  { todo, setTodo, addTodo, updateTodo }
) {
  const { todo } = useSelector((state: RootState) => state.todosReducer);
  const dispatch = useDispatch();
  return (
    <ListGroupItem>
      <Button onClick={() => dispatch(addTodo(todo))}>
        id="wd-add-todo-click" Add </Button>
      <Button onClick={() => dispatch(updateTodo(todo))}>
        id="wd-update-todo-click" Update </Button>
      <FormControl
        defaultValue={todo.title}
        onChange={(e) => dispatch(setTodo({ ...todo, title: e.target.value })))}/>
    </ListGroupItem>
  );
}
```

*// import useSelector, useDispatch
// to read/write to reducer
// reducer functions
// remove dependency from
// parent component

// retrieve todo from reducer
// create dispatch instance to
// invoke reducer functions

// wrap reducer functions
// with dispatch

// wrap reducer functions
// with dispatch*

Also reimplement the **TodoItem** component as shown below, using the reducer functions instead of the parameters.

app/Labs/Lab4/redux/todos/ToDoItem.tsx

```
import React from "react";
import { useDispatch } from "react-redux";
import { deleteTodo, setTodo } from "../todosReducer";
export default function TodoItem({ todo,
  deleteTodo, setTodo
}) {
  const dispatch = useDispatch();
  return (
    <ListGroupItem key={todo.id}>
      <Button onClick={() => dispatch(deleteTodo(todo.id))}>
        id="wd-delete-todo-click" Delete </Button>
      <Button onClick={() => dispatch(setTodo(todo))}>
        id="wd-set-todo-click" Edit </Button>
      {todo.title}
    </ListGroupItem>
  );
}
```

*// import useDispatch to invoke reducer
// functions deleteTodo and setTodo

// remove dependency with
// parent component
// create dispatch instance to invoke
// reducer functions

// wrap reducer functions with dispatch*

Reimplement the **TodoForm** and **TodoItem** components as shown above and update the **TodoList** component as shown below. Remove unnecessary dependencies and confirm that it works as before.

app/Labs/Lab4/redux/todos/ToDoList.tsx

```
import React from "react";
import TodoForm from "../TodoForm";
import TodoItem from "../TodoItem";
import { useSelector } from "react-redux";
import { RootState } from "../../store";
export default function TodoList() {
  const { todos } = useSelector((state: RootState) =>
    state.todosReducer);
  return (
    <div id="wd-todo-list-redux">
      <h2>Todo List</h2>
      <ListGroup>
        <TodoForm />
        {todos.map((todo: any) => (
          <TodoItem todo={todo} />
        ))}
      </ListGroup>
    </div>
  );
}
```

*// import useSelector to retrieve
// data from reducer

// extract todos from reducer and remove
// all other event handlers

// remove unnecessary attributes

// remove unnecessary attributes,
// but still pass the todo*

```

    </ListGroup>
    <hr/>
  </div>
);}

```

Now the todos are available to any component in the body of the **Provider**. To illustrate this, select the todos from within the **Lab4/ArrayStateVariable** component as shown below and confirm the todos display as expected.

app/Labs/Lab4/ArrayStateVariable.tsx

```

...
import { useSelector } from "react-redux";
import { RootState } from "../store";
export default function ArrayStateVariable() {
  const { todos } = useSelector((state: RootState) => state.todosReducer);
  ...
  return (
    <div id="wd-array-state-variables">
      <h2>Array State Variable</h2>
      ...
      <ListGroup>
        {todos.map((todo: any) => (
          <ListGroupItem key={todo.id}>
            {todo.title}
          </ListGroupItem>
        ))}
      </ListGroup>
      <hr />
      ...
    </div>);}

```

4.4 Maintaining State with React Context

React Context provides a way to share state across the component tree without manually passing props through every level, a problem known as "prop drilling". Unlike **Redux**, **React Context** is built directly into React.

4.4.1 Counter Context

To practice, let's reimplement the Counter using **React Context**. Create a **CounterContext.tsx** that defines the state and functions, then provides them to child components.

app/Labs/Lab4/react-context/counter/context.tsx

```

"use client";
import React, { createContext, useContext, useState, ReactNode } from "react";

// Define the context state
interface CounterContextState {
  count: number;
  increment: () => void;
  decrement: () => void;
}

// Create the context
const CounterContext = createContext<CounterContextState | undefined>(
  undefined,
);

// Create the provider component
export const CounterProvider = ({ children }: { children: ReactNode }) => {
  const [count, setCount] = useState(0);

  const increment = () => setCount((prev) => prev + 1);

```



```

const decrement = () => setCount((prev) => prev - 1);

const value: CounterContextState = {
  count,
  increment,
  decrement,
};

return (
  <CounterContext.Provider value={value}>{children}</CounterContext.Provider>
);
};

// Create a custom hook to use the counter context
export const useCounter = () => {
  const context = useContext(CounterContext);
  return context;
};

```

Create the **Counter Context** component below that uses the **useCounter** hook to access and update the counter state.

app/Labs/Lab4/react-context/counter/index.tsx

```

"use client";
import { useCounter } from "../context";

export default function CounterContext() {
  const { count, increment, decrement } = useCounter();

  return (
    <div id="wd-counter-context">
      <h2>Counter Context</h2>
      <h3>{count}</h3>
      <button onClick={increment} id="wd-counter-context-increment-click">
        Increment
      </button>
      <button onClick={decrement} id="wd-counter-context-decrement-click">
        Decrement
      </button>
      <hr />
    </div>
  );
}

```

In a new **React Context Examples** page, add the **Counter Context** component wrapped in a **Counter Provider** as shown below.

app/Labs/Lab4/react-context/page.tsx

```

"use client";
import { CounterProvider } from "../counter/context";
import CounterContext from "../counter";

export default function ReactContextExamples() {
  return (
    <div>
      <h1>React Context Examples</h1>
      <CounterProvider>
        <CounterContext />
      </CounterProvider>
    </div>
  );
}

```

In your **Lab 4** page, add a link to the **React Context** exercises and confirm that you can navigate to the **Counter Context** exercise and can increment and decrement the counter.

app/Labs/Lab4/page.tsx

```
import Link from "next/link";
export default function Lab4() {
  return (
    <div>
      <h2>Lab 4</h2>
      ....
      <Link href="/lab4/redux">Redux Examples</Link>
      <hr />
      <Link href="/lab4/react-context">React Context Examples</Link>
    </div>
  );
}
```

4.4.2 Implementing a React Context Todo List (On Your Own)

To practice creating stateful components with **React Context**, create a **Todo List** component like the one you created with **Redux**. The component should render as shown below, allow creating new todo items, updating their title, and deleting the todos.

Todo List

Learn Mongo	Update	Add
Learn React	Edit	Delete
Learn Node	Edit	Delete

Figure 4.4.2 - React Context Todo List

Your implementation should have at least the following files, but feel free to create additional files if you think you need more:

- **todosContext.ts** - Implements the context and provide that maintains the state for the todos
- **ReactContextTodoList.tsx** - Renders the todos as shown in Figure 4.4.2 allowing users to create, update, and delete todos

Add the **ReactContextTodoList** component to **ReactContextExamples** page and confirm you can navigate and interact with the **ReactContextTodoList** component.

4.5 Maintaining State with Zustand

Zustand is a lightweight, fast, and scalable state management library for **React** applications. Often described as "bearbones" state management, it provides a simple hook-based API that eliminates much of the boilerplate found in alternatives like **Redux**. You create a store using the **create** function from **Zustand**, and the resulting hook lets you read and update state anywhere in your app without needing a **Provider** component or complex setup. First, install **Zustand** with the following command from the root of your project.

```
npm install zustand
```

4.5.1 Implementing a Zustand Counter

Let's reimplement the same counter example to compare **Zustand** with equivalent implementations already implemented with **React Context** and **Redux**. Create the **Zustand store** shown below where we'll keep the state of the counter.

app/Labs/Lab4/zustand/counter/store.ts

```
import { create } from "zustand";

// Define the counter state
interface CounterState {
  count: number;
  increase: (by: number) => void;
  decrease: (by: number) => void;
  setCount: (count: number) => void;
  reset: () => void;
}

// Use create function to create a hook giving us access to the state
export const useCounterStore = create<CounterState>((set) => ({
  count: 0,
  increase: (by) => set((state) => ({ count: state.count + by })),
  decrease: (by) => set((state) => ({ count: state.count - by })),
  setCount: (count) => set({ count }),
  reset: () => set({ count: 0 }),
}));
```

Now create a component that uses the hook to access the state in the store.

app/Labs/Lab4/zustand/counter/index.tsx

```
"use client";
import { useCounterStore } from "../useCounterStore";

export default function ZustandCounter() {
  const { count, increase, decrease, setCount, reset } = useCounterStore(
    (state) => state,
  );

  return (
    <div className="m-2">
      <h1 className="text-3xl font-semibold leading-10 text-black">
        Zustand Counter
      </h1>
      Count: {count}
      <br />
      <button onClick={() => increase(1)}>Increase</button>
      <button onClick={() => decrease(1)}>Decrease</button>
      <button onClick={() => setCount(10)}>Set to 10</button>
      <button onClick={() => reset()}>Reset</button>
    </div>
  );
}
```

Create a page that uses the displays the **ZustandCounter** component.

app/Labs/Lab4/zustand/page.tsx

```
import ZustandCounter from "../counter";
export default function ZustandExamples() {
  return (
    <div>
      <h2>Zustand Examples</h2>
    </div>
  );
}
```

```

    <ZustandCounter />
  </div>
);
}

```

In the **Lab 4** page, add a link to the new **Zustand Examples** page. Confirm you can navigate to the new examples and update the **Zustand Counter** component.

app/Labs/Lab4/page.tsx

```

import Link from "next/link";
export default function Lab4() {
  return (
    <div>
      <h2>Lab 4</h2>
      ...
      <Link href="/lab4/redux">Redux Examples</Link>
      <hr />
      <Link href="/lab4/react-context">React Context Examples</Link>
      <hr />
      <Link href="/lab4/zustand">Zustand Examples</Link>
    </div>
  );
}

```

4.5.2 Implementing a Zustand Todo List (On Your Own)

To practice creating stateful components with **Zustand**, create a Todo List component like the one you created with Redux. The component should render as shown below, allow creating new todo items, updating their title, and deleting the todos.

Todo List

Learn Mongo	Update	Add
Learn React	Edit	Delete
Learn Node	Edit	Delete

Figure 4.5.2 - Zustand Todo List

Your implementation should have at least the following files, but feel free to create additional files if you think you need more:

- **useTodoStore.ts** - Implements the store that maintains the state for the todos
- **ZustandTodoList.tsx** - Renders the todos as shown in Figure 4.5.2 allowing users to create, update, and delete todos

Add the **ZustandTodoList** component to **ZustandExamples** page and confirm you can navigate and interact with the **ZustandTodoList** component.

4.6 Adding State to the Kambaz User Interface

The current **Kambaz** implementation reads data from a collection of objects we've combined into a **"Database"** containing **courses**, **modules**, **assignments**, and **grades**. The data is dynamically rendered in screens **Dashboard**, **Home**, **Module**, **Assignments**, and **Grades**. The data is currently static, and our **Kambaz** implementation is basically a set of functions that transform the data in the **Database** into a corresponding user interface. Since the data is static, the user interface is static as well. In this section we will use the component and application state management skills we learned in the **Labs** section to refactor the **Kambaz** application so we can create new **courses**, **modules** and **assignments**.

4.6.1 Adding State to the Kambaz Dashboard

The current **Dashboard** implementation renders a static array of courses. This section illustrates how to refactor the **Dashboard** to implement **CRUD** operations such as **creating** new courses, **retrieving** courses, **updating** existing course titles, and **deleting** courses. Import the **useState** hook and convert the **courses** constant into a state variable as shown below. Make these changes in the current implementation using the code below as an example.

app/(kambaz)/dashboard/page.tsx

```
"use client"
import { useState } from "react"; // add useState hook
import Link from "next/link";
import * as db from "../database";
export default function Dashboard() {
  const [courses, setCourses] = useState<any[]>(db.courses); // create courses state
  // variable and initialize
  // with database's courses
  return ( ... );
}
```

4.6.1.1 Creating New Courses

To create new courses, implement the **addNewCourse** function as shown below and add a new **Add** button that invokes the **addNewCourse** function to append a new course at the end of the **courses** array. The **addNewCourse** function overrides the **_id** property with a unique timestamp.

app/(kambaz)/dashboard/page.tsx

```
import { v4 as uuidv4 } from "uuid"; // install the uuid library if necessary
export default function Dashboard() {
  const [courses, setCourses] = useState<any[]>(db.courses);
  const course: any = { // create a course object with default values
    _id: "0", name: "New Course", number: "New Number",
    startDate: "2023-09-10", endDate: "2023-12-15",
    image: "/images/reactjs.jpg", description: "New Description"
  };
  const addNewCourse = () => {
    const newCourse = { ...course, _id: uuidv4() };
    setCourses([...courses, newCourse ]);
  };
  return (
    <div className="p-4" id="wd-dashboard">
      <h1 id="wd-dashboard-title">Dashboard</h1> <hr />
      <h5>New Course
        <button className="btn btn-primary float-end"
          id="wd-add-new-course-click"
          onClick={addNewCourse} > Add </button>
      </h5><hr />
    </div>
  );
}
```

```
...
</div>
);}
```

Dashboard

New Course

Add

Figure 4.4.1.1a - Creating New Courses

Confirm you can add new courses and the **Published Courses** counter reflects the new course. Modify the **img** tag so that it either renders a hardcoded image, e.g., **"/images/react.jpg"**, or renders the course's **image** property, but then you'll need to add image properties to **courses.json** on your own (not required). Alternatively each course can render as a different random color. Convert the **course** constant into a state variable as shown below. Add a form to edit the **course** state variable's **name**, and **description** properties. Confirm form shows values of the **course** state variable.

app/(kambaz)/dashboard/page.tsx

```
export default function Dashboard() {
  const [courses, setCourses] = useState<any[]>(db.courses);
  const [course, setCourse] = useState<any>({
    _id: "0", name: "New Course", number: "New Number",
    startDate: "2023-09-10", endDate: "2023-12-15",
    image: "/images/reactjs.jpg", description: "New Description"
  });
  const addNewCourse = () => { ... };
  return (
    <div id="wd-dashboard">
      <h1 id="wd-dashboard-title">Dashboard</h1> <hr />
      <h5>New Course ... </h5><br />
      <FormControl value={course.name} className="mb-2" />
      <FormControl value={course.description} rows={3}/>
      <hr /> ...
    </div>
  );}
```

// convert course into a state
// variable so we can change it
// and force a redraw of the UI

// add input element for each of
// fields in course state
// variable

New Course

Add

New Course

New Description

Figure 4.4.1.1b - Creating New Courses

Add **onChange** attributes to each of the input fields to update each of the fields using the **setCourse** mutator function, as shown below. Use your implementation of **Dashboard** and the code provided as an example. Confirm you can add new courses.

app/(kambaz)/dashboard/page.tsx

```
return (
  <div>
    <h1 id="wd-dashboard-title">Dashboard</h1> <hr />
    <h5>New Course ... </h5> <br />
    <FormControl value={course.name} className="mb-2"
      onChange={(e) => setCourse({ ...course, name: e.target.value }) } />
    <FormControl value={course.description} rows={3}>
```

```

    ...
    onChange={(e) => setCourse({ ...course, description: e.target.value }) } />
  </div>
);

```

4.6.1.2 Deleting a Course

Now let's implement deleting courses by adding **Delete** buttons to each of the courses. The buttons invoke a new **deleteCourse** function that accepts the ID of the course to remove. The function filters out the course from the **courses** array. Use the code below as an example to refactor your **Dashboard** component. Confirm that you can remove courses.

app/(kambaz)/dashboard/page.tsx

```

...
export default function Dashboard() {
  const [courses, setCourses] = useState<any[]>(db.courses);
  const [course, setCourse] = useState<any>({ ... });
  const addNewCourse = () => { ... };
  const deleteCourse = (courseId: string) => {
    setCourses(courses.filter((course) => course._id !== courseId));
  };
  return (
    <div id="wd-dashboard">
      <h1>Dashboard</h1>
      ...
      <div className="row">
        <div className="row row-cols-1 row-cols-md-5 g-4">
          {courses.map((course) => (
            ...
            <button className="btn btn-primary">
              Go </button>
            <button onClick={(event) => {
              event.preventDefault();
              deleteCourse(course._id);
            }} className="btn btn-danger float-end"
              id="wd-delete-course-click">
              Delete
            </button>
            ...
          ))}
        </div>
      </div>
    </div>
  );
}

```

// add deleteCourse event handler accepting
// ID of course to remove by filtering out
// the course by its ID

// add Delete button next to the course's
// name to invoke deleteCourse when clicked
// passing the course's ID and preventing
// the Link's default behavior to navigate
// to Course Screen

4.6.1.3 Editing a Course

Now let's implement editing an existing course by adding **Edit** buttons to each of the courses which invoke a new **setCourse** function that copies the current course into the **course** state variable, displaying the course in the form so you can edit it. Refactor your **Dashboard** component using the code below as an example. Confirm that clicking **Edit** on a course, copies the course into the form.

app/(kambaz)/dashboard/page.tsx

```

<button id="wd-edit-course-click"
  onClick={(event) => {
    event.preventDefault();
    setCourse(course);
  }}
  className="btn btn-warning me-2 float-end" >
  Edit
</button>

```

// next to the Delete button
// add an Edit button to copy the course
// to be edited into the form so we can
// edit it. Prevent default to navigate
// to Course screen

Add an **Update** button to the form so that the selected course can be updated with the values in the edited fields. Use the code below as an example. Confirm you can select, and then edit the selected course. Confirm that clicking **Update** actually updates the original course's **name** and **description**.

```
app/(kambaz)/dashboard/page.tsx

...
export default function Dashboard() {
  const [courses, setCourses] = useState<any[]>(db.courses);
  const [course, setCourse] = useState<any>({ ... });
  const updateCourse = () => {
    setCourses(
      courses.map((c) => {
        if (c._id === course._id) {
          return course;
        } else {
          return c;
        }
      })
    );
  };
  return (
    <div id="wd-dashboard">
      <h1 id="wd-dashboard-title">Dashboard</h1> <hr />
      <h5> New Course
        <button className="btn btn-primary float-end"
          onClick={addNewCourse} id="wd-add-new-course-click">
          Add </button>
        <button className="btn btn-warning float-end me-2"
          onClick={updateCourse} id="wd-update-course-click">
          Update </button>
      </h5>
      ...
    </div>
  );
}
```

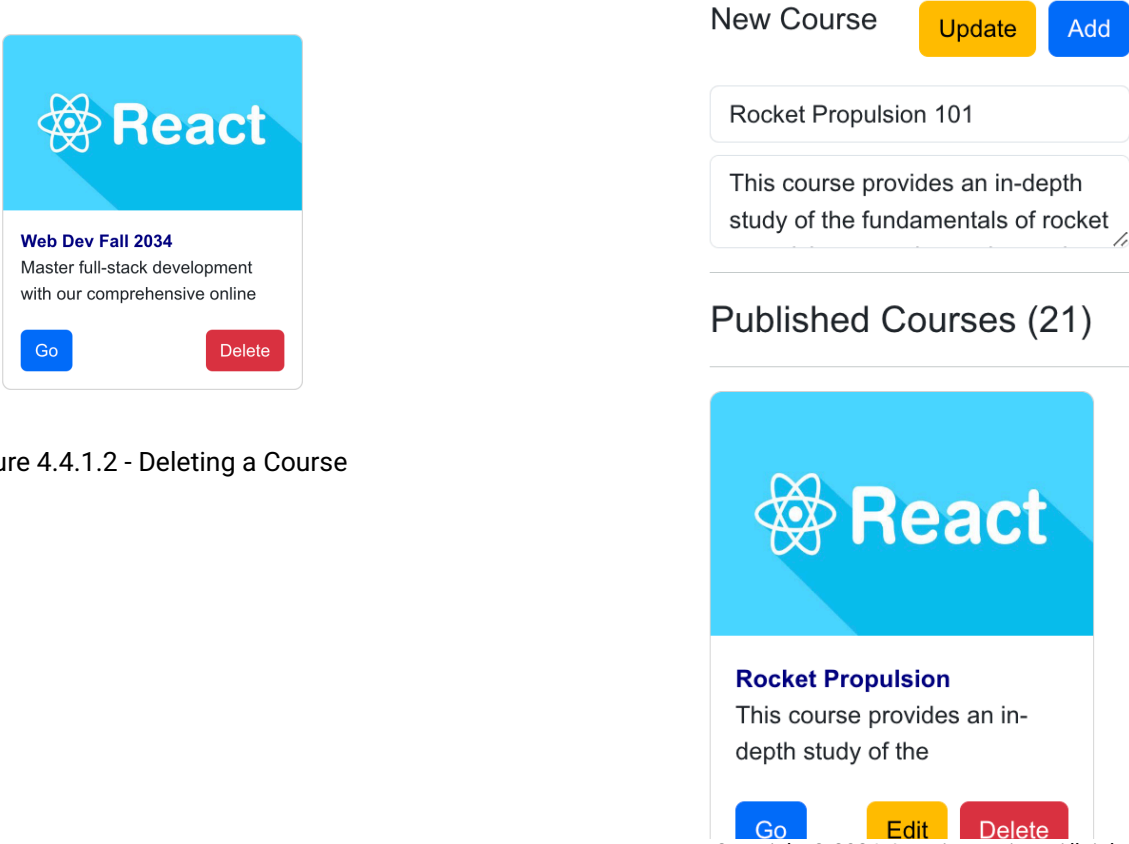


Figure 4.4.1.2 - Deleting a Course

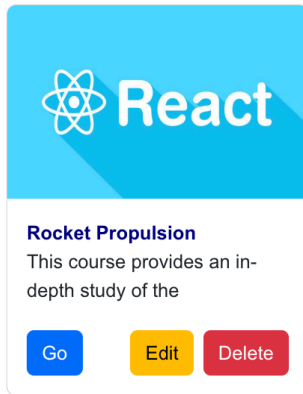


Figure 4.4.1.3a - Editing a Course

Figure 4.4.1.3b - Editing a Course

4.6.2 Courses Reducer

The **Dashboard** component seems to be working fine, but the courses it is creating, deleting, and updating can not be used outside of the component. This is a problem because the **Courses** screen would want to be able to render the new courses, but it doesn't have access to the **courses** state variable in the **Dashboard**. To fix this we need to either add redux so all courses are available everywhere, or move the **courses** state variable up to a component that is parent to both the **Dashboard** and the **Courses**. Let's use **Redux** to share the courses among several screens. In the **Courses** folder, declare the following reducer. Note that the reducer functions are very similar to the ones implemented in the **Dashboard** screen, but adapted to use **Redux** syntax.

`app/(kambaz)/courses/reducer.ts`

```
import { createSlice } from "@reduxjs/toolkit";
import { courses } from "../database";
import { v4 as uuidv4 } from "uuid";
const initialState = {
  courses: courses,
};
const coursesSlice = createSlice({
  name: "courses",
  initialState,
  reducers: {
    addNewCourse: (state, { payload: course }) => {
      const newCourse = { ...course, _id: uuidv4() };
      state.courses = [...state.courses, newCourse] as any;
    },
    deleteCourse: (state, { payload: courseId }) => {
      state.courses = state.courses.filter(
        (course: any) => course._id !== courseId
      );
    },
    updateCourse: (state, { payload: course }) => {
      state.courses = state.courses.map((c: any) =>
        c._id === course._id ? course : c
      ) as any;
    },
    setCourses: (state, { payload: courses }) => {
      state.courses = courses;
    },
  },
});
export const { addNewCourse, deleteCourse, updateCourse } =
  coursesSlice.actions;
export default coursesSlice.reducer;
```

Let's create a store that can hold all **Kambaz** related data starting with the courses as shown below.

app/(kambaz)/store.ts

```
import { configureStore } from "@reduxjs/toolkit";
import coursesReducer from "../courses/reducer";
const store = configureStore({
  reducer: { coursesReducer },
});
export type RootState = ReturnType<typeof store.getState>;
export default store;
```

Import and use the **store** and **Provider** to provide all **Kambaz** related data to all pages and components as shown below.

app/(kambaz)/Layout.tsx

```
"use client";
import { ReactNode } from "react";
import KambazNavigation from "../Navigation";
import "../styles.css";
import store from "../store";
import { Provider } from "react-redux";
export default function KambazLayout({
  children,
}: Readonly<{ children: ReactNode }>) {
  return (
    <Provider store={store}>
      <div className="d-flex" id="wd-kambaz">
        <div>
          <KambazNavigation />
        </div>
        <div className="flex-fill ps-3 wd-main-content-offset">{children}</div>
      </div>
    </Provider>
  );
}
```

4.6.3 Refactoring Dashboard to Use Redux

Now that we have the courses available to all **Kambaz** pages and components through the **Redux** store, let's refactor the **Dashboard** page to use the **courses** reducer instead of the local state. Import the **Redux** hooks to retrieve reducer data and send dispatch it data. Import the reducer functions from the **courses** reducer. Retrieve the **courses** from the store, remove all local state mutator functions and refactor all calls to the reducer functions by wrapping them with **dispatch**.

app/(kambaz)/dashboard/page.tsx

```
import { useDispatch, useSelector } from "react-redux";
import { addNewCourse, deleteCourse, updateCourse, setCourses } from "../courses/reducer";
import { RootState } from "../store";
export default function Dashboard() {
  const { courses } = useSelector((state: RootState) => state.coursesReducer);
  const dispatch = useDispatch();
  const [course, setCourse] = useState<any>({ ... });
  const addNewCourse = (course: any) => { }
  const deleteCourse = (courseId: string) => { }
  const updateCourse = () => { }
  return (
    ...
    <button ... onClick={() => dispatch(addNewCourse(course))} >
      Add
    </button>
    <button ... onClick={() => dispatch(updateCourse(course))} >
      Update
    </button>
    <button ... onClick={(event) => {
```

```

        event.preventDefault();
        dispatch(deleteCourse(course._id));
    }} >
Delete
</button>
);}

```

4.6.4 Courses Screen

Now that we have the **courses** declared in the **Kambaz** store, we can share them with the **Courses** screen component by retrieving the same courses from the same courses reducer. The **Courses** component destructs the courses from the reducer and then finds the course by the **cid** path parameter searching through the **courses** state variable. Refactor your **Courses** layout component as suggested below and confirm you can navigate to new courses created in the **Dashboard**. Also, when the **FaAlignJustify** icon next to the course name is clicked, toggle the visibility of the **CourseNavigation** sidebar. That is, if the user clicks

app/(kambaz)/courses/[cid]/layout.tsx

```

"use client";
import { ReactNode } from "react";
import CourseNavigation from "../Navigation";
import { useSelector } from "react-redux";
import { useParams } from "next/navigation";
import { RootState } from "../../store";
export default function CoursesLayout({ children }: { children: ReactNode }) {
  const { cid } = useParams();
  const { courses } = useSelector((state: RootState) => state.coursesReducer);
  const course = courses.find((course: any) => course._id === cid);
  return (
    <div id="wd-courses">
      <h2>
        <FaAlignJustify className="me-4 fs-4 mb-1" />
        {course?.name}
      </h2>
      <hr />
      <div className="d-flex">
        <div>
          <CourseNavigation />
        </div>
        <div className="flex-fill">{children}</div>
      </div>
    </div>
  );
}

```

4.6.5 Toggling the Course Navigation Sidebar

On the left of the course name in the **Courses** page there's a **≡ sandwich** symbol to toggle the courses navigation sidebar. Implement the toggling behavior so that when users click on the sandwich, the courses navigation hides, and if it's clicked again, the navigation shows again.

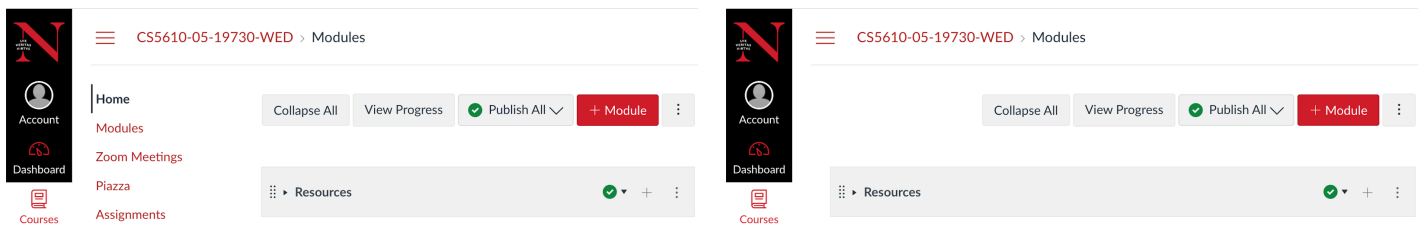


Figure 4.4.2.1 Toggling the Course Navigation Sidebar

4.6.6 Adding State Management to the Modules Screen

Now let's do the same with **Modules** refactoring the component adding state variables so that we can create, update, and remove modules. We'll discover the same limitation we had with **courses**, i.e., we won't be able to share new modules outside the **Modules** screen. But instead of moving the modules state variable and functions to a shared common parent component, we'll instead use **Redux** to make the modules available throughout the application. The screenshot here on the right is for illustration purposes only. Reuse the HTML and CSS from previous chapters to style your modules. Refactor your **Modules** implementation by converting the **modules** array into a state variable as shown below. Confirm **Modules** renders as expected. Styling shown here is for illustration purposes. Use your HTML and CSS from previous chapters to style the modules.

`app/(kambaz)/courses/[cid]/modules/page.tsx`

```
import { useState } from "react";
import { useParams } from "next/navigation";
import * as db from "../../database";
export default function Modules() {
  const { cid } = useParams();
  const [modules, setModules] = useState<any[]>(db.modules);
  return ( ... );
}
```

4.6.6.1 Creating a Module

Let's create a modal dialog where users can edit the name of a new module based on [Bootstrap's modal component](#). The **ModuleEditor** component below pops up when you click the red **+ Module** button in the **Modules** and **Home** screens. You can type the name of the new module in an input field. As you type the name of the module, the **setModuleName** updates the module name, and clicking on the **Add Module** button calls the **addModule** function which actually adds the module.

`app/(kambaz)/courses/[cid]/modules/ModuleEditor.tsx`

```
import { Modal, FormControl, Button } from "react-bootstrap";
export default function ModuleEditor({ show, handleClose, dialogTitle, moduleName, setModuleName, addModule, }: {
  show: boolean; handleClose: () => void; dialogTitle: string; moduleName: string; setModuleName: (name: string) => void;
  addModule: () => void; }) {
  return (
    <Modal show={show} onHide={handleClose}>
      <Modal.Header closeButton>
        <Modal.Title>{dialogTitle}</Modal.Title>
      </Modal.Header>
      <Modal.Body>
        <FormControl value={moduleName}
          onChange={(e) => { setModuleName(e.target.value); }} />
      </Modal.Body>
      <Modal.Footer>
        <Button variant="secondary" onClick={handleClose}> Cancel </Button>
        <Button variant="primary"
          onClick={() => {
            addModule();
            handleClose();
          }} > Add Module </Button>
      </Modal.Footer>
    </Modal>
  );
}
```

The **+ Module** button was implemented in the **ModulesControls** in a prior chapter. Let's refactor it so that it displays the **ModuleEditor** dialog when clicked.

`app/(kambaz)/courses/[cid]/modules/modulesControls.tsx`

```
import ModuleEditor from "../ModuleEditor";
export default function ModulesControls(
{ moduleName, setModuleName, addModule }:
{ moduleName: string; setModuleName: (title: string) => void; addModule: () => void; }) {
  const [show, setShow] = useState(false);
  const handleClose = () => setShow(false);
  const handleShow = () => setShow(true);
  return (
    <div id="wd-modules-controls" className="text-nowrap">
      <Button variant="danger" onClick={handleShow}>
        <FaPlus className="position-relative me-2" style={{ bottom: "1px" }} />
        Module
      </Button>
      ...
      <ModuleEditor show={show} handleClose={handleClose} dialogTitle="Add Module"
        moduleName={moduleName} setModuleName={setModuleName} addModule={addModule} />
    </div>
  );
};
```

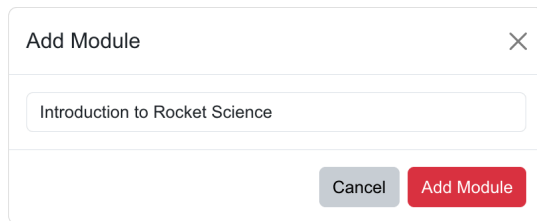


Figure 4.4.3.1a - Module Editor



Figure 4.4.3.1b - Modules Controls

In the **Modules** screen, declare a **moduleName** state variable that keeps track of the module name edited in the **ModuleEditor** dialog. The **addModule** function below creates a new module instance with the **moduleName** as the name and appends it to the end of the **modules** state variable. The **setModuleName** and **addModule** functions are passed to the **ModulesControls** component which will in turn pass them to the **ModuleEditor** dialog. The **ModuleEditor** dialog will invoke the **setModuleName** function when editing the module name in the dialog text field. The dialog will invoke the **addModule** function when the **Add Module** button is clicked. Confirm you can add modules.

app/(kambaz)/courses/[cid]/modules/page.tsx

```
import { v4 as uuidv4 } from "uuid";
export default function Modules() {
  const { cid } = useParams();
  const [modules, setModules] = useState<any[]>(db.modules);
  const [moduleName, setModuleName] = useState("");
  const addModule = () => {
    setModules([ ...modules, { _id: uuidv4(), name: moduleName, course: cid, lessons: [] } ]);
    setModuleName("");
  };
  return (
    <div className="wd-modules">
      <ModulesControls setModuleName={setModuleName} moduleName={moduleName} addModule={addModule} />
      ...
    </div>
  );
};
```

4.6.6.2 Deleting a Module

To delete modules, let's add a **trashcan** icon to the **ModuleControlButtons** implement in an earlier chapter. We'll pass it a **deleteModule** we can call when clicking the **trashcan** and also pass the ID of the module to be deleted **moduleId**.

app/(kambaz)/courses/[cid]/modules/ModuleControlButtons.tsx

```
import { FaTrash } from "react-icons/fa";
export default function ModuleControlButtons(
  { moduleId, deleteModule }: { moduleId: string; deleteModule: (moduleId: string) => void; } ) {
  return (
    <div className="float-end">
      <FaTrash className="text-danger me-2 mb-1" onClick={() => deleteModule(moduleId)} />
      <GreenCheckmark />
      <BsPlus className="fs-1" />
      <IoEllipsisVertical className="fs-4" />
    </div> );}
```

⋮ Introduction to Rocket Science   + ⋮

4.4.3.2 Deleting a Module

In the **Modules** screen, let's implement **deleteModule** function that removes a module by its **ID** and then pass the function to the **ModuleControlButtons** component along **moduleId** to be removed. Confirm you can remove modules.

app/(kambaz)/courses/[cid]/modules/page.tsx

```
export default function Modules() {
  const addModule = () => { ... };
  const deleteModule = (moduleId: string) => {
    setModules(modules.filter((m) => m._id !== moduleId));
  };
  return (
    <div className="wd-modules">
      ...
      <ModuleControlButtons
        moduleId={module._id}
        deleteModule={deleteModule} />
      ...
    </div>
  );
}
```

4.6.6.3 Editing a Module

In **ModuleControlButtons**, add a pencil icon as shown below. Clicking the icon should call **editModule** with the **moduleId** of the module we want to edit.

app/(kambaz)/courses/[cid]/modules/ModuleControlButtons.tsx

```
import { FaTrash } from "react-icons/fa";
import { FaPencil } from "react-icons/fa6";
export default function ModuleControlButtons({ moduleId, deleteModule, editModule }: {
  moduleId: string; deleteModule: (moduleId: string) => void;
  editModule: (moduleId: string) => void; }) {
  return (
    <div className="float-end">
      <FaPencil onClick={() => editModule(moduleId)} className="text-primary me-3" />
      <FaTrash ... />
      <GreenCheckmark />
      <BsPlus className="fs-1" />
      <IoEllipsisVertical className="fs-4" />
    </div>
  );}
```

⋮ Fundamentals of Aerodynamics    + ⋮

⋮ Fundamentals of Aerodynamics 101    + ⋮

Figure 4.4.3.3a - Editing a Module

Figure 4.4.3.3b - Editing a Module

In the **Modules** component, implement functions **editModule** and **updateModule** as shown below. Pass the **editModule** function to the **ModuleControlsButtons** component so that when the **pencil** icon is clicked, it will invoke the **editModule** to set the module's **editing** field to true. The **updateModule** accepts a **module** object and updates the corresponding **module** object in the **modules** array. If the **module**'s **editing** field is not set (false), then the **module**'s **name** is displayed. But if the **pencil** is clicked, the **module**'s **editing** field is set to true and instead of the **module**'s **name**, an input field is displayed so the **name** can be edited using the **updateModule** function. If the **Enter** key is pressed, the **module**'s **editing** field is set to false, and then the editing field is hidden and the **module**'s name is shown again. Confirm you can edit the names of the modules.

app/(kambaz)/courses/[cid]/modules/page.tsx

```
export default function Modules() {
  const { cid } = useParams();
  const [modules, setModules] = useState<any[]>(db.modules);
  const [moduleName, setModuleName] = useState("");
  const addModule = () => { ... }
  const deleteModule = (moduleId: string) => { ... }
  const editModule = (moduleId: string) => {
    setModules(modules.map((m) => (m._id === moduleId ? { ...m, editing: true } : m)));
  };
  const updateModule = (module: any) => {
    setModules(modules.map((m) => (m._id === module._id ? module : m)));
  };
  return (
    <div className="wd-title p-3 ps-2 bg-secondary">
      <BsGripVertical className="me-2 fs-3" />
      {!module.editing && module.name}
      {module.editing && (
        <FormControl className="w-50 d-inline-block"
          onChange={(e) => updateModule({ ...module, name: e.target.value })}
          onKeyDown={(e) => {
            if (e.key === "Enter") {
              updateModule({ ...module, editing: false });
            }
          }}
          defaultValue={module.name}/>
      )}
      <ModuleControlButtons
        moduleId={module._id}
        deleteModule={deleteModule}
        editModule={editModule}/>
    </div>
  );
}
```

// set the module's editing flag
 // to true so that we can display
 // the input field to edit name
 // update any field(s) of a
 // module

// show name if not editing
 // show input field if editing
 // when typing edit the module's
 // name
 // if "Enter" key is
 // pressed then set editing
 // field to false so we
 // hide the text field

// pass editModule function to
 // so if pencil is clicked we can
 // set editing to true

4.6.6.4 Module Reducer

The **Modules** component seems to be working fine. We can create new modules, edit modules, and remove modules, BUT, it suffers a major flaw. Those new modules and edits can't be used outside the confines of the **Modules** component even though we would want to display the same list of modules elsewhere such as the **Home** screen. We could use the same approach as we did for the **Dashboard**, by moving the state variables and functions to a higher level component that could share the state with other components. Instead we're going to use **Redux** this time to practice application level state management. To start, create the **reducer.tsx** shown below containing the **modules** state variables as well as the **addModule**, **deleteModule**, **updateModule**, and **editModule** functions reimplemented in the **reducers** property.

app/(kambaz)/courses/[cid]/modules/reducer.ts

```
import { createSlice } from "@reduxjs/toolkit";
import { modules } from "../../database";

// import createSlice
// import modules from database
```

```

import { v4 as uuidv4 } from "uuid";
const initialState = {
  modules: modules,
};
const modulesSlice = createSlice({
  name: "modules",
  initialState,
  reducers: {
    addModule: (state, { payload: module }) => {
      const newModule: any = {
        _id: uuidv4(),
        lessons: [],
        name: module.name,
        course: module.course,
      };
      state.modules = [...state.modules, newModule] as any;
    },
    deleteModule: (state, { payload: moduleId }) => {
      state.modules = state.modules.filter(
        (m: any) => m._id !== moduleId);
    },
    updateModule: (state, { payload: module }) => {
      state.modules = state.modules.map((m: any) =>
        m._id === module._id ? module : m
      ) as any;
    },
    editModule: (state, { payload: moduleId }) => {
      state.modules = state.modules.map((m: any) =>
        m._id === moduleId ? { ...m, editing: true } : m
      ) as any;
    },
  },
});
export const { addModule, deleteModule, updateModule, editModule } =
  modulesSlice.actions;
export default modulesSlice.reducer;

```

*// create reducer's initial state with
 // default modules copied from database*

*// create slice
 // name the slice
 // set initial state
 // declare reducer functions
 // new module is in action.payload
 // update modules in state adding new module
 // at beginning of array. Override _id with
 // timestamp*

*// module's ID to delete is in action.payload
 // filter out module to delete*

*// module to update is in action.payload
 // replace module whose ID matches
 // action.payload._id*

// select the module to edit

// export all reducer functions

// export reducer

Add the new modules reducer to the **Kambaz store** created earlier as shown below.

app/(kambaz)/store.ts

```

import { configureStore } from "@reduxjs/toolkit";
import coursesReducer from "../courses/reducer";
import modulesReducer from "../courses/[cid]/modules/reducer";
const store = configureStore({
  reducer: {
    coursesReducer,
    modulesReducer,
  },
});
export type RootState = ReturnType<typeof store.getState>;
export default store;

```

Reimplement the **Modules** by removing the state variables and functions, and replacing them with selectors, dispatchers, and reducer functions as shown below. Confirm you can still add, remove, and edit modules as before. Also confirm the modules still work in the **Home** screen.

app/(kambaz)/courses/[cid]/modules/page.tsx

```

import { addModule, editModule, updateModule, deleteModule }
  from "../reducer";
import { useSelector, useDispatch } from "react-redux";
import { RootState } from "../../store";
export default function Modules() {
  const { cid } = useParams();
  const [moduleName, setModuleName] = useState("");
  const { modules } = useSelector((state: RootState) => state.modulesReducer);

```

*// import reducer functions to add,
 // delete, and update modules
 // import useSelector and useDispatch*

// retrieve modules state variables


```

const dispatch = useDispatch();
return (
  <div className="wd-modules">
    <ModulesControls moduleName={moduleName} setModuleName={setModuleName}>
      addModule={() => {
        dispatch(addModule({ name: moduleName, course: cid }));
        setModuleName("");
      }} />
    <ListGroup id="wd-modules" className="rounded-0">
      {modules
        .filter((module: any) => module.course === cid)
        .map((module: any) => (
          <!module.editing && module.name>
          < module.editing && (
            <FormControl className="w-50 d-inline-block"
              onChange={(e) =>
                dispatch(
                  updateModule({ ...module, name: e.target.value })
                )
              }
              onKeyDown={(e) => {
                if (e.key === "Enter") {
                  dispatch(updateModule({ ...module, editing: false }));
                }
              }}
              defaultValue={module.name} />
            <ModuleControlButtons moduleId={module._id}>
              deleteModule={(moduleId) => {
                dispatch(deleteModule(moduleId));
              }}
              editModule={(moduleId) => dispatch(editModule(moduleId))} />
          </ModuleControlButtons>
        ))}
      </ListGroup>
    </ListGroup>
  </div>
);
}

```

// get dispatch to call reducer
// functions

// wrap reducer functions with
// dispatch clear module name

// wrap reducer functions with
// dispatch

// wrap reducer functions with
// dispatch

// wrap reducer
// functions with
// dispatch

4.6.7 Account Screens

The **Account Screens** provide users access to their personal information and all related data such as courses they are enrolled in and courses they might be teaching. Users use the **Sign In** screen to identify themselves and access their **Profile** screen to view their personal information. This section describes refactoring the **Signin** and **Profile** screens to confirm a user's identity and display their personal information.

4.6.7.1 Account Reducer

Implement an **account reducer** to keep track of the currently signed in user and share it across the entire application. Implement the **account reducer** as shown below and then add it to the **Kambaz** store.

```
app/(kambaz)/account/reducer.ts      app/(kambaz)/store.ts

import { createSlice } from "@reduxjs/toolkit";
const initialState = {
  currentUser: null,
};
const accountSlice = createSlice({
  name: "account",
  initialState,
  reducers: {
    setCurrentUser: (state, action) => {
      state.currentUser = action.payload;
    },
  },
});
export const { setCurrentUser } = accountSlice.actions;
```

```
import { configureStore } from "@reduxjs/toolkit";
import coursesReducer from "../courses/reducer";
import modulesReducer from "../courses/[cid]/modules/reducer";
import accountReducer from "../account/reducer";
const store = configureStore({
  reducer: {
    coursesReducer,
    modulesReducer,
    accountReducer,
  },
});
export type RootState = ReturnType<typeof store.getState>;
export default store;
```

```
export default accountSlice.reducer;
```

4.6.7.2 Signin

Refactor the **Signin** screen by adding a **credentials** state variable for users to enter their credentials. When users click the **Sign In** button, search for a user with the credentials. If there's a user that matches, store it in the reducer by **dispatching** it to the **Account reducer** using the **setCurrentUser** reducer function. Ignore the **Sign In** attempt if there's no match. After signing in, navigate to the **Dashboard** as shown below. Confirm that signing in navigates to the **Dashboard** only if valid credentials are used.

app/(kambaz)/account/signin/page.tsx

```
"use client";
import Link from "next/link";
import { redirect } from "next/navigation";
import { setCurrentUser } from "../reducer";
import { useDispatch } from "react-redux";
import { useState } from "react";
import * as db from "../../database";
import { FormControl, Button } from "react-bootstrap";

export default function Signin() {
  const [credentials, setCredentials] = useState<any>({});
  const dispatch = useDispatch();
  const signin = () => {
    const user = db.users.find(
      (u: any) =>
        u.username === credentials.username &&
        u.password === credentials.password
    );
    if (!user) return;
    dispatch(setCurrentUser(user));
    redirect("/dashboard");
  };

  return (
    <div id="wd-signin-screen">
      <h1>Sign in</h1>
      <FormControl
        defaultValue={credentials.username}
        onChange={(e) => setCredentials({ ...credentials, username: e.target.value })}
        className="mb-2" placeholder="username" id="wd-username" />
      <FormControl
        defaultValue={credentials.password}
        onChange={(e) => setCredentials({ ...credentials, password: e.target.value })}
        className="mb-2" placeholder="password" type="password" id="wd-password" />
      <Button
        onClick={signin}
        id="wd-signin-btn"
        className="w-100" > Sign in </Button>
      <Link id="wd-signup-link" href="/account/signup"> Sign up </Link>
    </div>
  );
}
```

4.6.7.3 Dashboard

Now that the current user is stored in the **Account reducer**, the **Dashboard** can filter the courses and only display the courses in which the current user is enrolled in. Refactor the **Dashboard** so that it only shows the courses the current user is enrolled in as shown below. Sign in as different users and confirm that the **Dashboard** only displays the courses a user is enrolled in. Note that new courses added will not render now since enrollments would also need to be modified. This will be addressed in later chapters.

app/(kambaz)/dashboard/page.tsx

```
import * as db from "../../database";
import { RootState } from "../store";
...
export default function Dashboard() {
```

```

const { courses } = useSelector((state: RootState) => state.coursesReducer);
const { currentUser } = useSelector((state: RootState) => state.accountReducer);
const { enrollments } = db;
const dispatch = useDispatch();
return(
  ...
  {courses
    .filter((course) =>
      enrollments.some(
        (enrollment) =>
          enrollment.user === currentUser._id &&
          enrollment.course === course._id
        )
      )
    .map((course) => (
      <div className="wd-dashboard-course col" style={{ width: "300px" }} >
        ...
      </div>
    ))
  )}
  ...
);}

```

4.6.7.4 Account Navigation

Users can use the **Account Navigation** sidebar to navigate between the **Account Screens** *Signin*, *Signup*, and *Profile*, but not all screens should be available if there's a user is logged in or not. Reimplement the **Account Navigation** sidebar so that it hides the *Signin* and *Signup* navigation links if a user is already signed in, and hides the *Profile* link if a user is not yet signed in.

app/(kambaz)/account/Navigation.tsx

```

"use client";
import Link from "next/link";
import { usePathname } from "next/navigation";
import { Nav, NavItem, NavLink } from "react-bootstrap";
import { useSelector } from "react-redux";
import { RootState } from "../store";
export default function AccountNavigation() {
  const { currentUser } = useSelector((state: RootState) => state.accountReducer);
  const links = currentUser ? ["profile"] : ["signin", "signup"];
  const pathname = usePathname();
  return (
    <Nav variant="pills">
      {links.map((link) => (
        <NavItem key={link}>
          <NavLink as={Link} href={link} active={pathname.endsWith(link)}>
            {link} </NavLink> </NavItem>
          )
        )}
    </Nav>
  );}

```

Also refactor the **Account** screen so that the default screen is *Signin* if no one is signed in yet, and *Profile* if someone is already signed in. Confirm that the **Account Navigation** links are *Sign In* and *Sign Up* if no one is signed in yet, and *Profile* if someone is already signed in. Also confirm that clicking the **Account** link in the **Kambaz Navigation** sidebar displays the *Signin* screen if no one is signed in yet, and displays the *Profile* screen if someone is already signed in.

app/(kambaz)/account/page.tsx

```

"use client";
import { useSelector } from "react-redux";
import { RootState } from "../store";
import { redirect } from "next/navigation";
export default function AccountPage() {
  const { currentUser } = useSelector((state: RootState) => state.accountReducer);
  if (!currentUser) {
    redirect("/account/signin");
  } else {

```

```

    redirect("/account/profile");
  }
}

```

4.6.7.5 Profile

The **Profile** screen displays the current user's personal information. Refactor the **Profile** screen to retrieve the current user from the **Account reducer**. If there's no **currentUser**, then the screen should redirect to the **Signin** screen. If there's a **currentUser** then the **Profile** screen should populate a form with the user's information. If the current user clicks a **Sign Out** button, then the current user should be nulled and navigate to the **Signin** screen. Refactor the **Profile** screen as shown below.

app/(kambaz)/account/profile/page.tsx

```

"use client";
import { redirect } from "next/navigation";
import { useState, useEffect } from "react";
import { useSelector, useDispatch } from "react-redux";
import { setCurrentUser } from "../reducer";
import { RootState } from "../../store";
import { Button, FormControl } from "react-bootstrap";
export default function Profile() {
  const [profile, setProfile] = useState<any>({});
  const dispatch = useDispatch();
  const { currentUser } = useSelector((state: RootState) => state.accountReducer);
  const fetchProfile = () => {
    if (!currentUser) return redirect("/account/signin");
    setProfile(currentUser);
  };
  const signout = () => {
    dispatch(setCurrentUser(null));
    redirect("/account/signin");
  };
  useEffect(() => {
    fetchProfile();
  }, []);
  return (
    <div className="wd-profile-screen">
      <h3>Profile</h3>
      {profile && (
        <div>
          <FormControl id="wd-username" className="mb-2"
            defaultValue={profile.username}
            onChange={(e) => setProfile({ ...profile, username: e.target.value })} />
          <FormControl id="wd-password" className="mb-2"
            defaultValue={profile.password}
            onChange={(e) => setProfile({ ...profile, password: e.target.value })} />
          <FormControl id="wd-firstname" className="mb-2"
            defaultValue={profile.firstName}
            onChange={(e) => setProfile({ ...profile, firstName: e.target.value })} />
          <FormControl id="wd-lastname" className="mb-2"
            defaultValue={profile.lastName}
            onChange={(e) => setProfile({ ...profile, lastName: e.target.value })} />
          <FormControl id="wd-dob" className="mb-2" type="date"
            defaultValue={profile.dob}
            onChange={(e) => setProfile({ ...profile, dob: e.target.value })} />
          <FormControl id="wd-email" className="mb-2"
            defaultValue={profile.email}
            onChange={(e) => setProfile({ ...profile, email: e.target.value })} />
          <select className="form-control mb-2" id="wd-role"
            onChange={(e) => setProfile({ ...profile, role: e.target.value })} >
            <option value="USER">User</option>
            <option value="ADMIN">Admin</option>
            <option value="FACULTY">Faculty</option>{" "}
            <option value="STUDENT">Student</option>
          </select>
          <Button onClick={signout} className="w-100 mb-2" id="wd-signout-btn">
            Sign out
          </Button>
        </div>
      )}
    </div>
  );
}

```

```

    </Button>
  </div>
) }
</div>
};

```

4.6.8 Assignments (On Your Own)

After completing the **Dashboard**, **Courses**, and **Modules**, refactor the **Assignments** and **AssignmentEditor** screens so that faculty can create, update, and remove assignments as described in this section. Students can only view assignments.

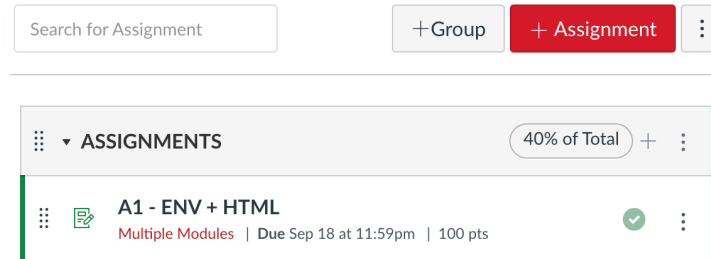


Figure 4.4.5 - Assignments

4.6.8.1 Assignments Reducer

Following **modules/reducer.ts** as an example, create an **Assignments/reducer.ts** in **app/(kambaz)/courses/assignments/** initialized with **db.assignments**. Implement reducer functions such as **addAssignment**, **deleteAssignment**, **updateAssignment**, and any other functions as needed. Add the new reducer to the store in **kambaz/store/index.ts** to add the assignments to the **Kambaz** application state.

4.6.8.2 Creating an Assignment

Refactor your **Assignments** screen as follows

- Clicking the **+ Assignment** button navigates to the **AssignmentEditor** screen
- The **AssignmentEditor** should allow editing at least the following fields: **name**, **description**, **points**, **due date**, **available from date**, and **available until date**.
- Clicking **Save** creates the new assignment and adds it to the **assignments** array state variable, navigates to the **Assignments** screen, which must now contain the newly created assignment.
- Clicking **Cancel** does not create the new assignment, and navigates back to the **Assignments** screen, without the new assignment.

4.6.8.3 Editing an Assignment

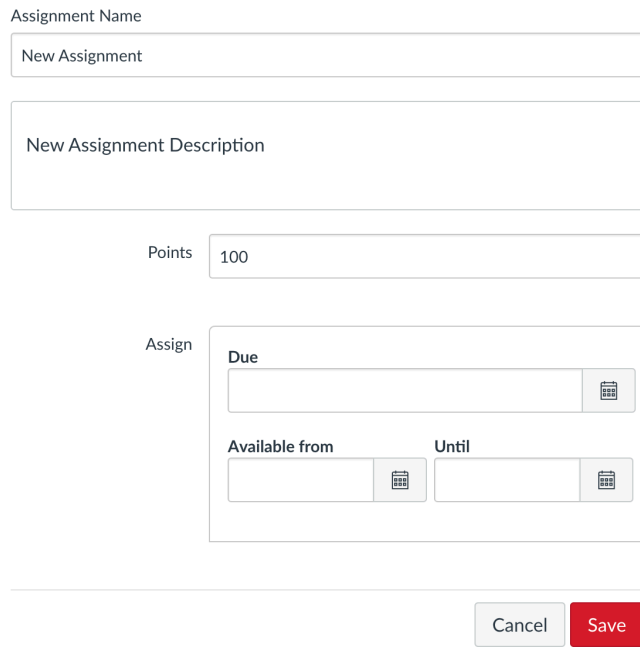
Refactor the **AssignmentsEditor** component as follows

- Clicking on an assignment in the **Assignments** screen navigates to the **AssignmentsEditor** screen, displaying the assignment's **name**, **description**, **points**, **due date**, **available from date**, and **available until date** of the corresponding assignment.
- The **AssignmentsEditor** screen should allow editing the same fields listed earlier for corresponding assignment.
- Clicking **Save** updates the assignment's fields and navigates back to the **Assignments** screen with the updated assignment values.
- Clicking **Cancel** does not update the assignment, and navigates back to the **Assignments** screen which shows the assignments unchanged.

4.6.8.4 Deleting an Assignment

Refactor the **Assignments** component as follows

- Using the example of deleting modules, add a **Delete** button or **trash** icon to the right of each assignment.
- Clicking **Delete** on an assignment pops up a dialog asking if you are sure you want to remove the assignment.
- Clicking **Yes** or **Ok**, dismisses the dialog, removes the assignment, and updates the **Assignments** screen without the deleted assignment.
- Clicking **No** or **Cancel**, dismisses the dialog without removing the assignment



The form is titled "Assignment Name" and contains a text input field with the placeholder "New Assignment". Below this is a larger text area with the placeholder "New Assignment Description". To the right of the description area is a "Points" label and a text input field containing the value "100". Below the points field is an "Assign" label and a date picker component. The date picker has a "Due" field, an "Available from" field, and an "Until" field, each with a calendar icon. At the bottom right of the form are "Cancel" and "Save" buttons.

Figure 4.4.5.2 - Creating and Editing Assignments

4.6.9 Enrollments (On Your Own)

Currently the **Dashboard** screen allows **Faculty** to **Add**, **Delete**, **Edit**, and **Update** courses, as well as navigate to the course's content. Other types of the users can only view the list of the courses they are enrolled in. Refactor the **Dashboard** screen so that there's a new blue **Enrollments** button at the top right of the screen. Clicking the **Enrollments** button displays all the the courses. Clicking it again only shows the courses a user is enrolled in. Courses that the user is enrolled in should provide a red **Unenroll** button and courses that the user is not enrolled in should provide a green **Enroll** button. When a user clicks the **Unenroll** or **Enroll** button the enrollment status must actually change and the buttons should toggle to reflect the new state. If a user signs out, and then signs in again, the enrollment choices should still persist. If a user refreshes or reloads the page, the new enrollments are lost. Protect the route to a course so that only users enrolled in that course can navigate to the course, and stay in the **Dashboard** screen otherwise. Create new or modify existing reducers and store as needed.

4.7 Deliverables

1. In the same React application created in earlier chapters, **kambaz-next-js**, complete all the exercises described in this document.
2. In a branch called **a4**, add, commit and push the source code of the React application **kambaz-next-js** to the same remote source repository in **GitHub.com** created in an earlier chapter. Here's an example of how to add, commit and push your code

```
$ git checkout -b a4
$ git add .
$ git commit -am "a4 Redux"
$ git push
```

3. Deploy the **a4** branch to the same **Vercel** project created in an earlier chapter. Configure Vercel to deploy all branches to separate URLs. From your Vercel's dashboard go to **Site settings > Build & deploy > Branches > Branch deploys** and select **All**. Now each time you commit to a branch, the application will be available at a URL that contains the name of the branch
4. Make sure **labs/page.tsx** contains a **TOC.tsx** that references each of the labs and Kambaz. Add a link to your repository in GitHub.
5. In **labs/page.tsx**, add your full name: first name first, and last name second. Use the same name as in Canvas.
6. As a deliverable in **Canvas**, submit the URL to the **a4** branch deployment of your React application running on Vercel.